

CAHIER DES CHARGES

Application web de gestion et de réservation pour un réseau de clubs de padel multi-sites

KANGOUTE Azoumanan

EPHEC — Bachelier en Informatique

Cours de Programmation Web (PDW) et Bases de données (SGBD)

Lien github Projet Padel : <https://github.com/Zoom225/padellAout.git>

1. Procédure de lancement complète

J'ai un fichier readme.md qui resume tout pour la compilation du projet

Cette procédure complète la partie exploitation. Elle permet de lancer la base, le backend et le frontend dans le bon ordre.

1. Depuis la racine du projet : lancer PostgreSQL docker compose up - d

2. Lancer le backend Spring Boot
./mvnw spring-boot:run

3. Dans un autre terminal : lancer Angular cd frontend
npm install
npm start

Élément	Valeur du projet
Base de données	PostgreSQL 15, base padelService, utilisateur padel
Port PostgreSQL hôte	5440 -> 5432 conteneur
Backend	localhost:8081
Frontend	localhost:4200
Swagger	/swagger-ui.html

Table des matières

1. Présentation du projet	4
1.1. Contexte et objet du document	4
1.2. Problématique métier	5
1.3. Objectifs	6

1.4. Acteurs et profils utilisateurs	7
1.5. Périmètre fonctionnel et exclusions	10
2. Description fonctionnelle détaillée	13
2.1. Authentification et gestion des sessions	13
2.2. Gestion des membres	16
2.3. Sites, terrains et jours de fermeture	18
2.4. Cycle de vie d'un match	21
2.5. Réservation d'une place	24
2.6. Paiement et confirmation	26
2.7. Annulations et remboursements	29
2.8. Automatisations quotidiennes	31
3. Règles métier implémentées	33
3.1. Gestion des sites	33
3.2. Gestion des terrains	35
3.3. Gestion des jours de fermeture	38
3.4. Tarification d'un match	41
3.5. Fenêtre de réservation	43
3.6. Disponibilité d'un créneau	46
3.7. Ouverture et fermeture des sites	49
3.8. Périmètre des membres SITE	52
3.9. Capacité et statut d'un match	54
3.10. Réservation	57
3.11. Paiement et solde	60
3.12. Annulation	63
3.13. Traitements automatiques et pénalités	65
3.14. Authentification et sécurité	68
3.15. Périmètre et propriété des ressources	71
3.16. Validation des saisies	74
4. Modèle de données	76
4.1. Entités et attributs	76
4.2. Relations	78
4.3. Énumérations de statut	79
5. Architecture technique	80
5.1. Organisation en couches	80
5.2. Sécurité applicative	81
5.3. Frontend	82
5.4. Base de données et déploiement	83

5.5. Tests automatisés	84
6. Aperçu de l'API REST	85
7. Exploitation	86
8. Limites connues et perspectives	87
9. Conclusion	88
10. Compléments techniques	89
10.1. Compatibilité avec le projet livré	
10.2. Architecture Backend	
10.3. Architecture Frontend	
10.4. Flux Frontend / Backend	
10.5. Preuves des règles métier	
10.6. Sécurité JWT	
10.7. Gestion des erreurs	
10.8. Relations JPA	
10.9. Catalogue API REST	
10.10. Tests automatisés	
10.11. Explication du projet	
10.12. Compléments des règles métier	
10.13. Limites techniques	
10.14. Quelques explications	
10.15. Checklist finale	90

1. Présentation du projet

1.1. Contexte et objet du document

PadelMultiple est une application web destinée à un opérateur gérant plusieurs clubs de padel répartis sur des sites géographiques distincts. Chaque site dispose de ses propres terrains, de ses horaires d'ouverture, d'une durée standard de match et d'une durée de pause imposée entre deux matchs successifs sur un même terrain. L'application permet aux joueurs (membres) de créer des matchs ou de rejoindre des matchs existants, de réserver une place et de la payer ; elle permet aux administrateurs de gérer les sites, les terrains, les jours de fermeture et les membres.

La version actuelle du projet est constituée d'un backend Spring Boot situé dans le package `com.padell.padell` et d'un frontend Angular situé dans `frontend/src/app`. Le backend centralise la sécurité, les règles métier et l'accès aux données PostgreSQL. Le frontend assure l'affichage, la navigation, les formulaires, la gestion locale des sessions et les appels HTTP vers l'API REST.

Ce cahier des charges décrit en détail les besoins fonctionnels, les règles métier et les choix techniques de l'application. Il a la particularité d'avoir été rédigé à partir de l'analyse directe du code source livré (entités, services métier, contrôleurs REST, configuration de sécurité et tests). Chaque règle métier présentée correspond donc à un comportement réellement implémenté et vérifiable dans le code, ce qui garantit la cohérence entre le présent document et l'application.

Le document est organisé en plusieurs parties complémentaires. Les premiers chapitres Présentent le contexte, les acteurs, les fonctionnalités, les règles métier, le modèle de données et l'architecture technique. Les chapitres suivants détaillent l'architecture Réelle du backend Spring Boot et du frontend Angular, les flux entre les couches, la sécurité JWT, la gestion des erreurs, les relations JPA, les endpoints REST, les tests automatisés et les Procédures d'exploitation. Cette organisation permet de relier les besoins fonctionnels aux preuves techniques présentes dans le projet livré.

1.2. Problématique métier

La gestion d'un réseau multi-sites de padel soulève plusieurs difficultés que l'application doit résoudre de manière automatisée et fiable.

D'abord, les réservations doivent être réparties sur plusieurs sites aux caractéristiques différentes (horaires, terrains, durée de match) sans jamais provoquer de double réservation d'un même terrain sur des créneaux qui se chevauchent, et en respectant une pause de remise en état entre deux matchs.

Ensuite, l'opérateur souhaite distinguer plusieurs catégories de membres dont les droits de réservation diffèrent, afin de privilégier certains profils dans l'accès aux créneaux. L'application doit aussi tenir compte des jours de fermeture, qu'ils concernent un seul site ou l'ensemble du réseau.

Par ailleurs, un match se joue à quatre joueurs et son coût doit être partagé équitablement. Un organisateur peut ouvrir un match en cercle restreint (privé) ou ouvert à tous (public). Pour éviter qu'un match privé resté incomplet n'immobilise inutilement un terrain, l'application doit automatiquement le rendre public à l'approche de l'échéance, tout en responsabilisant l'organisateur défaillant. Enfin, le système doit gérer les paiements, les sommes dues et les pénalités, afin qu'un membre ne respectant pas ses engagements soit temporairement empêché de

réserver. L'ensemble de ces règles doit être appliqué côté serveur, pour qu'aucune ne puisse être contournée depuis l'interface.

1.3. Objectifs

L'application poursuit les objectifs suivants :

- permettre la réservation en ligne de terrains de padel sur plusieurs sites, avec consultation des terrains et des matchs publics disponibles ;
- différencier les droits de réservation selon le profil du membre (GLOBAL, SITE, LIBRE) ;

- encadrer la création de matchs (durée fixe, créneau libre, pause respectée, horaires d'ouverture, jours de fermeture) ;
- gérer la participation à un match dans la limite de quatre joueurs, en confirmant la place uniquement au paiement (principe « premier payé, premier servi ») ;
- automatiser la conversion des matchs privés incomplets en matchs publics et l'application de pénalités ;
- gérer les soldes dus et bloquer les membres en situation d'impayé ou de pénalité ;
- offrir une administration des sites, terrains, jours de fermeture et membres, avec un contrôle d'accès limitant chaque administrateur à son périmètre ;
- garantir la sécurité de l'ensemble par une authentification à base de jetons et une séparation stricte des rôles.

1.4. Acteurs et profils utilisateurs

L'application distingue deux familles d'utilisateurs : les membres (joueurs) et les administrateurs. Les membres se répartissent en trois profils, identifiés par le préfixe de leur matricule, qui déterminent leur périmètre de réservation et leur horizon d'anticipation. Les administrateurs se répartissent en deux rôles selon l'étendue de leur pouvoir de gestion.

Profil / Rôle	Identifiant	Périmètre	Particularité
Membre GLOBAL	Matricule G + 4 chiffres et mot de passe	Tous les sites	Réservation jusqu'à 21 jours à l'avance
Membre SITE	Matricule S + 5 chiffres et mot de passe	Son site de rattachement	Réservation jusqu'à 14 jours à l'avance
Membre LIBRE	Matricule L + 5 chiffres et mot de passe	Tous les sites	Réservation jusqu'à 5 jours à l'avance
Admin GLOBAL	E-mail / mot de passe	Tout le réseau	Gère tous les sites, terrains, membres, matchs

Admin SITE	E-mail / mot de passe	Son site uniquement	Gestion limitée, contrôlée côté serveur
------------	-----------------------	---------------------	--

Les profils métier GLOBAL, SITE et LIBRE ne doivent pas être confondus avec les rôles techniques utilisés par Spring Security. Tous les joueurs authentifiés possèdent l'autorité technique `ROLE_MEMBER`, tandis que les administrateurs possèdent `ROLE_ADMIN`. Les valeurs GLOBAL, SITE et LIBRE déterminent ensuite le périmètre métier de l'utilisateur.

Pourquoi

Cela évite de confondre :

- Membre GLOBAL ;
- Administrateur GLOBAL ;
- Rôle Spring Security.

Preuve dans le projet

- `JwtAuthenticationFilter.java`
- `SecurityConfig.java`
- `TypeMembre.java`
- `TypeAdministrateur.java`

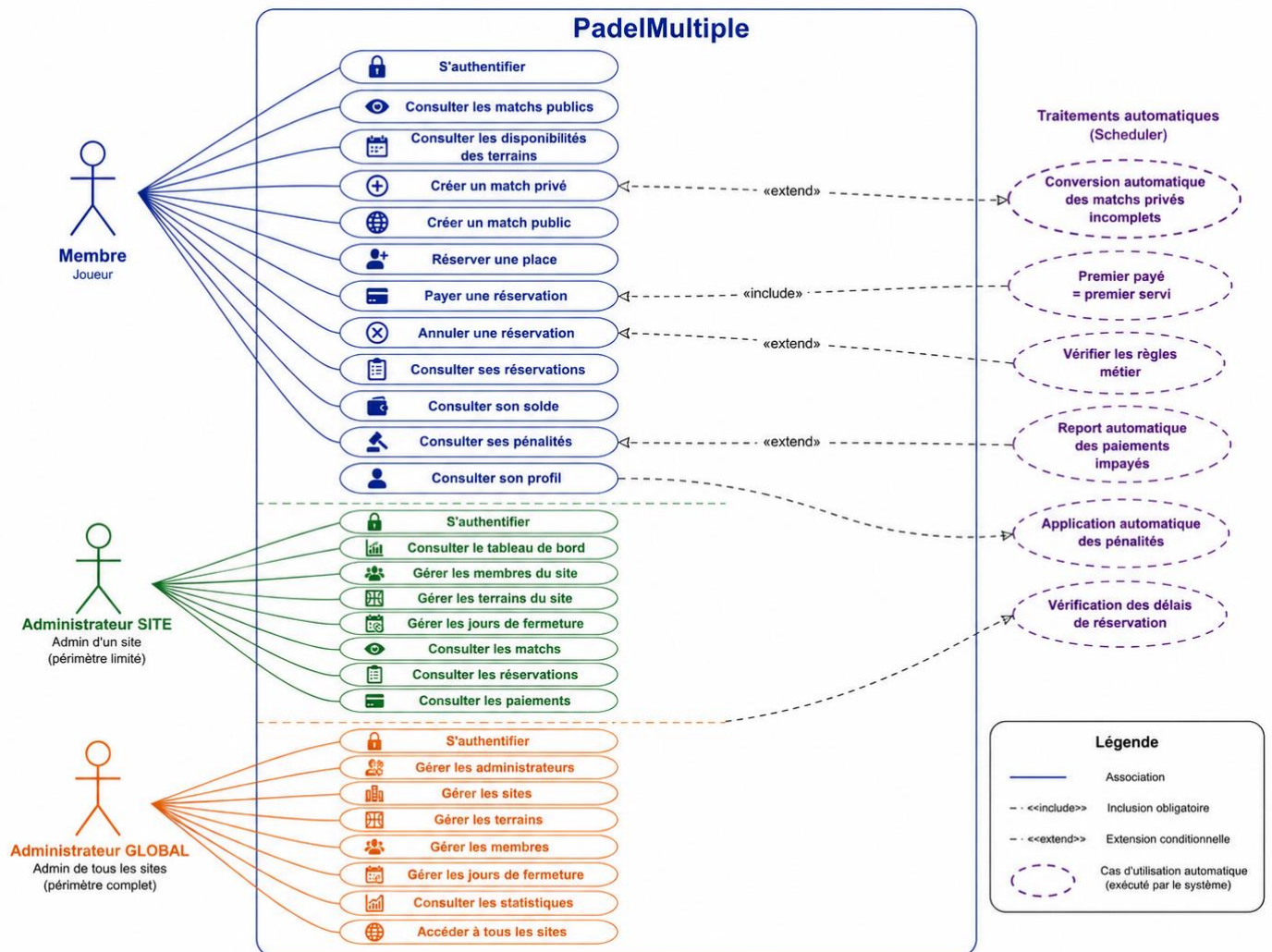


Figure 1 — Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les principales fonctionnalités métier offertes par l'application PadelMultiple pour les différents acteurs (membre, administrateur SITE et administrateur GLOBAL). La version actuelle du projet intègre également des fonctionnalités techniques, telles que la gestion multilingue de l'interface, la séparation des sessions entre membres et administrateurs, ainsi que la protection des routes Angular par des Guards. Ces mécanismes techniques ne constituent pas des cas d'utilisation métier, mais contribuent à renforcer la sécurité, la maintenabilité et l'expérience utilisateur de l'application.

1.5. Périmètre fonctionnel et exclusions

Le périmètre fonctionnel couvre la gestion complète d'un réseau de clubs de padel multi-sites. Il comprend la gestion des sites, des terrains, des jours de fermeture, des membres et des

administrateurs, ainsi que l'authentification sécurisée des utilisateurs. L'application permet la création de matchs publics et privés, la réservation de places, le paiement selon le principe « premier payé = premier servi », la gestion des soldes, des pénalités et des annulations. Elle intègre également des traitements automatiques, tels que la conversion des matchs privés incomplets en matchs publics et l'application des pénalités prévues par les règles métier. Le système repose sur une API REST sécurisée par JWT, une base de données PostgreSQL et une architecture séparant clairement le frontend Angular et le backend Spring Boot. Enfin, le projet comprend des tests automatisés garantissant le bon fonctionnement des principales fonctionnalités. Sont explicitement hors périmètre :

- le paiement bancaire réel par carte ou prestataire externe : le paiement reste une opération applicative interne dont seul le statut est suivi ;
- l'envoi de notifications par e-mail, SMS ou en temps réel ;
- la récupération de mot de passe oublié ;

- la gestion d'un classement ou d'un score des matchs joués ;
- une application mobile native, seule une application web étant prévue.

2. Description fonctionnelle détaillée

2.1. Authentification et gestion des sessions

L'application gère deux mécanismes d'authentification distincts. L'administrateur se connecte avec son adresse e-mail et son mot de passe : le mot de passe saisi est comparé au hachage BCrypt stocké en base, et en cas de succès un jeton JWT est émis. Ce jeton contient l'identité de l'administrateur et son rôle (GLOBAL ou SITE), et la réponse de connexion renvoie également le nom, le prénom, le rôle et, le cas échéant, l'identifiant du site de rattachement. En cas d'identifiants erronés, le système renvoie un message neutre « Identifiants invalides » qui ne précise pas si c'est l'e-mail ou le mot de passe qui est en cause, afin de ne pas faciliter une tentative malveillante.

Le membre se connecte au moyen de son matricule. Quel que soit l'utilisateur, le jeton émis porte une autorité technique qui distingue les deux univers : un membre reçoit l'autorité `ROLE_MEMBER` et un administrateur `ROLE_ADMIN`. Cette distinction est essentielle : elle garantit qu'un membre ne pourra jamais atteindre une route réservée à l'administration, et inversement, indépendamment du libellé GLOBAL/SITE qui sert au périmètre métier. L'API est sans état : aucune session n'est conservée côté serveur et chaque requête protégée doit présenter son jeton.

Côté interface, l'application gère séparément la session administrateur et la session membre. Pour éviter qu'une requête ne parte avec le mauvais jeton lorsque les deux sessions coexistent sur le même navigateur, la connexion à l'une efface systématiquement l'autre.

Le frontend met en œuvre cette gestion des sessions au moyen de services dédiés aux membres et aux administrateurs. Les informations d'authentification sont conservées localement après la connexion et automatiquement utilisées lors des appels sécurisés vers l'API REST. Un intercepteur HTTP ajoute le jeton JWT dans l'en-tête `Authorization` de chaque requête protégée, tandis que des Guards Angular contrôlent l'accès aux pages réservées selon le type d'utilisateur connecté. Cette architecture renforce la sécurité de navigation tout en séparant clairement les responsabilités entre l'interface utilisateur et le backend.

2.2. Gestion des membres

La création d'un membre est une opération d'administration : il n'existe pas d'inscription publique libre par l'API. Lorsqu'un administrateur crée un membre, le système valide d'abord que le format du matricule correspond au profil choisi, puis vérifie que le matricule et l'adresse e-mail ne sont pas déjà utilisés, et impose qu'un membre de profil SITE soit rattaché à un site existant. Le solde du nouveau membre est initialisé à zéro.

Dans la version actuelle du projet, la création d'un membre respecte davantage le principe de séparation des responsabilités. Le contrôleur reçoit la requête HTTP et la transmet aux

services métier spécialisés. La création du membre, les validations métier, le hachage du mot de passe, le rattachement éventuel à un site ainsi que les contrôles d'accès sont réalisés dans des services dédiés. Cette organisation améliore la lisibilité du code, facilite les tests unitaires et limite les responsabilités des contrôleurs.

La consultation des informations d'un membre — sa fiche, son solde, l'existence d'une pénalité — est réservée au membre concerné lui-même ou à un

administrateur. La liste complète des membres, la modification et la suppression sont réservées aux administrateurs, un administrateur de profil SITE ne pouvant agir que sur les membres rattachés à son propre site.

Les contrôles d'autorisation sont centralisés afin de garantir qu'un administrateur SITE ne puisse accéder qu'aux ressources de son propre site, tandis qu'un administrateur GLOBAL dispose d'un accès à l'ensemble des sites. Cette vérification est réalisée avant toute opération sensible afin de garantir le respect des droits d'accès.

2.3. Sites, terrains et jours de fermeture

Un site décrit un club : son nom, son adresse, son heure d'ouverture et de fermeture, la durée standard d'un match en minutes, la durée de pause imposée entre deux matchs et une année civile de référence. Un terrain appartient à un seul site et porte un prix. La consultation des sites et des terrains est publique, car elle est nécessaire au membre pour choisir un terrain au moment de créer un match ; en revanche, leur création, modification et suppression sont réservées aux administrateurs, dans la limite du périmètre de chacun.

Les jours de fermeture sont entièrement gérés par l'application. Un jour de fermeture porte une date, une raison facultative et un indicateur précisant s'il est global (il concerne alors l'ensemble du réseau) ou propre à un site donné. L'API permet de lister, créer et supprimer ces jours de fermeture, la gestion étant réservée aux administrateurs. Un jour de fermeture a un effet concret : il interdit la création et la modification de tout match prévu à cette date sur le site concerné.

Les contrôles relatifs aux horaires d'ouverture, aux jours de fermeture et à la disponibilité des terrains sont centralisés dans la couche métier du backend. Avant toute création ou modification d'un match, le système vérifie automatiquement que le terrain appartient bien au site sélectionné, que le créneau demandé respecte les horaires d'ouverture, qu'aucun jour de fermeture n'est applicable et qu'aucun autre match ne provoque un chevauchement sur le même terrain. Cette validation garantit la cohérence des réservations et évite les conflits de planification.

2.4. Cycle de vie d'un match

La création d'un match est l'opération la plus encadrée de l'application. Un membre choisit un terrain, une date et une heure de début, et un type de match (privé ou public). Le système procède alors à une série de vérifications, dans un ordre précis, avant d'accepter la création.

Il contrôle d'abord que l'organisateur n'a pas de solde impayé, puis qu'il n'a pas de pénalité active : dans l'un ou l'autre cas, la création est refusée. Il vérifie ensuite que la date demandée respecte l'horizon de réservation autorisé pour le profil du membre, et qu'un membre de profil SITE ne tente pas de réserver sur un autre site que le sien. Il calcule alors l'heure de fin du match en ajoutant à l'heure de début la durée standard configurée pour le site. Il s'assure que la date n'est pas un jour de fermeture, que le créneau tient dans les heures d'ouverture du site, et qu'aucun autre match non annulé ne chevauche ce créneau sur le même terrain, pause comprise.

Si toutes ces conditions sont remplies, le match est enregistré avec le statut « planifié », un nombre de joueurs confirmés initialisé à zéro, un prix total de 60 € et un prix par joueur de 15 €. Immédiatement après, le système crée automatiquement, pour l'organisateur lui-même, une réservation en attente et le paiement en attente correspondant : l'organisateur occupe ainsi une place comme n'importe quel autre joueur, place qui sera confirmée lorsqu'il aura payé sa part.

Un match peut ensuite être modifié par son organisateur tant qu'il n'est pas annulé : la modification refait les mêmes contrôles de fermeture, d'horaires et de disponibilité du créneau. Un match peut également être annulé par son organisateur ; un match déjà annulé ne peut plus être modifié ni annulé une seconde fois. Le statut d'un match évolue automatiquement : il passe à « complet » dès qu'il atteint quatre joueurs confirmés, et revient à « planifié » si un joueur confirmé se désiste et que le nombre repasse sous quatre.

2.5. Réservation d'une place

Pour rejoindre un match, un membre effectue une réservation. Le système vérifie successivement que le match n'est ni complet ni annulé, que le membre n'y est pas déjà inscrit, qu'il n'a pas de pénalité active et pas de solde impayé. La distinction entre match privé et public est ici déterminante : dans un match privé, seul l'organisateur

peut inscrire d'autres joueurs ; dans un match public, un membre ne peut réserver que pour lui-même. Enfin, un membre de profil SITE ne peut réserver que sur un match organisé sur son propre site.

Lorsque toutes ces conditions sont satisfaites, la réservation est enregistrée avec le statut « en attente

», et un paiement en attente du montant de la part par joueur lui est immédiatement associé. À ce stade, la place n'est pas encore acquise : elle ne le devient qu'au paiement.

2.6. Paiement et confirmation

Le paiement d'une réservation matérialise le principe « premier payé, premier servi », central dans l'application. Seul le membre concerné par la réservation peut la payer. Le système refuse le paiement

d'une réservation annulée, d'un paiement déjà effectué ou d'un paiement annulé. Au moment du paiement, il recharge l'état du match sous verrou afin d'éviter tout conflit entre deux paiements simultanés, vérifie que la réservation est bien en attente, puis contrôle que le match n'a pas déjà atteint quatre joueurs : si la dernière place vient d'être prise par un autre membre qui a payé en premier, le paiement est refusé et la place déclarée indisponible.

Si la place est encore disponible, le système prend en compte un éventuel solde dû par le membre : ce solde est ajouté au montant réglé puis remis à zéro, de sorte que régler une nouvelle réservation solde aussi les dettes antérieures. Le paiement est alors marqué comme effectué avec sa date, et la réservation est confirmée — ce qui incrémente le nombre de joueurs du match et, au quatrième joueur, le fait passer au statut « complet ».

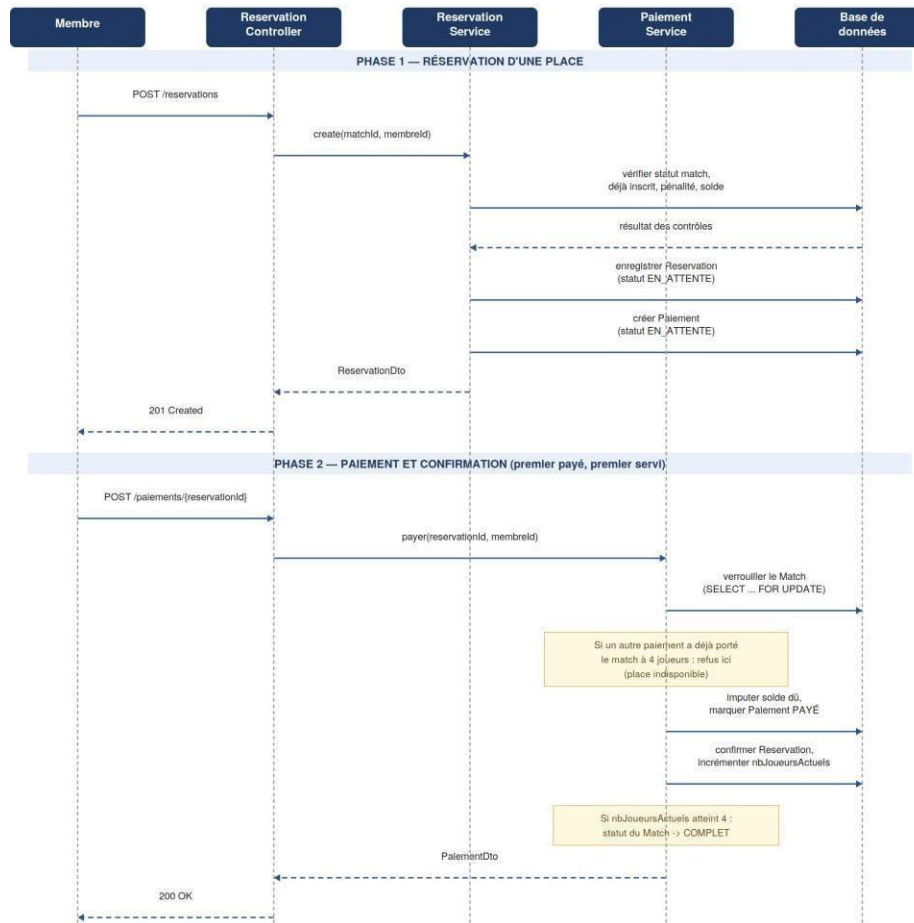


Figure 2 — Diagramme de séquence : réservation puis paiement (premier payé, premier servi)

2.7. Annulations et remboursements

L'annulation d'une réservation distingue selon que la place avait été confirmée ou

non. Si la réservation était seulement en attente, son annulation ne modifie pas le nombre de joueurs du match, puisque la place n'avait jamais été comptabilisée. Si la réservation était confirmée, son annulation décrémente le nombre de joueurs, ce qui peut faire repasser le match de « complet » à « planifié ». Dans tous les cas, le paiement associé est mis à jour de façon cohérente : un paiement déjà effectué passe au statut « remboursé », tandis qu'un paiement encore en attente passe au statut « annulé ».

2.8. Automatisations quotidiennes

Deux traitements s'exécutent automatiquement chaque jour, sans intervention humaine. Le premier, déclenché à minuit, recherche les matchs privés prévus pour le lendemain qui sont encore au statut « planifié » et n'ont pas atteint quatre joueurs ; il les convertit en matchs publics, enregistre la date de conversion et applique à leur organisateur une pénalité d'une semaine. L'objectif est double : libérer la place pour d'autres joueurs en ouvrant le match, et responsabiliser l'organisateur qui n'a pas réuni son groupe à temps.

Le second traitement recherche les réservations restées impayées pour les matchs du lendemain. Il annule chacune de ces réservations et reporte le montant de la part correspondante sur le solde de l'organisateur du match. Ainsi, l'organisateur supporte financièrement les places que les joueurs inscrits n'ont finalement pas payées, et ce solde dû viendra bloquer ses futures créations de match et réservations tant qu'il n'est pas réglé.

3. Règles métier implémentées

Ce chapitre détaille, règle par règle, la logique métier effectivement codée dans l'application. Chaque règle est identifiée par un code (RM-xx), accompagnée de son énoncé et d'une explication de son intention et de son fonctionnement. La numérotation est volontairement fine : un comportement distinct correspond à une règle distincte, afin de couvrir l'ensemble des contrôles présents dans le code et de faciliter la vérification de la correspondance entre le présent document et l'implémentation.

Les règles métier sont principalement implémentées dans la couche **Service** du backend Spring Boot. Les contrôleurs se limitent à recevoir les requêtes HTTP et à déléguer leur traitement, tandis que les repositories assurent uniquement l'accès aux données. Cette organisation respecte une architecture en couches, facilite les tests unitaires et garantit que toutes les validations métier sont centralisées avant toute modification de la base de données.

Gestion des sites

Les règles présentées dans cette section correspondent aux comportements réellement implémentés dans le projet. Chaque règle est appliquée par un ou plusieurs services métier et contribue à garantir la cohérence des données ainsi que le respect des contraintes fonctionnelles définies pour l'application.

L'application gère un ensemble de sites au moyen d'opérations complètes de création, consultation, modification et suppression.

Un site regroupe son nom, son adresse, ses heures d'ouverture et de fermeture, la durée standard d'un match, la durée de pause entre deux matchs et une année civile de référence. La modification met à jour l'ensemble de ces attributs sur le site existant. La suppression d'un site entraîne, par cascade, celle de ses terrains et de ses jours de fermeture rattachés.

RM-02 — Consultation publique des sites

La consultation des sites est accessible sans authentification.

Cette ouverture est nécessaire pour qu'un visiteur ou un membre puisse parcourir les clubs et leurs caractéristiques avant de choisir un terrain. La création, la modification et la suppression restent en revanche réservées aux administrateurs.

3.1. Gestion des terrains

RM-03 — Rattachement d'un terrain à un site

Tout terrain est créé en étant rattaché à un site existant.

À la création d'un terrain, le site cible est récupéré et associé au terrain ; si le site n'existe pas, l'opération échoue. Un terrain ne peut donc exister sans site de rattachement, ce qui garantit la cohérence de la hiérarchie site-terrain.

RM-04 — Prix d'un terrain

Un terrain porte un prix, utilisé comme base de tarification des matchs qui s'y déroulent.

Le prix est un attribut du terrain. Il sert de prix total du match (voir RM-09) et, divisé par le nombre de joueurs, de part individuelle. La gestion du prix se fait au travers des opérations d'administration sur les terrains.

RM-05 — Consultation des terrains et des terrains par site

La consultation des terrains est publique, et la liste des terrains d'un site donné est disponible après vérification de l'existence du site.

La lecture des terrains est ouverte, comme celle des sites, pour permettre le choix d'un terrain lors de la création d'un match. Lorsqu'on demande les terrains d'un site précis, le système vérifie d'abord que le site existe avant de renvoyer ses terrains, de façon à signaler clairement un identifiant erroné.

Les règles relatives aux terrains sont appliquées par la couche métier avant toute création ou modification. Le système vérifie notamment le rattachement du terrain à un site existant ainsi que la cohérence des informations enregistrées. Les opérations de gestion (création, modification et suppression) sont réservées aux administrateurs autorisés, tandis que la consultation des terrains reste accessible selon les droits définis par l'application. Cette organisation garantit l'intégrité des données et prépare les validations effectuées lors de la création des matchs.

Les fonctionnalités de gestion des terrains sont accessibles depuis l'interface d'administration Angular, qui communique avec le backend au moyen de services API dédiés. Les contrôleurs reçoivent les requêtes HTTP, les services appliquent les règles métier et les repositories assurent la persistance des données dans PostgreSQL.

3.2. Gestion des jours de fermeture**RM-06 — Fermeture globale ou propre à un site**

Un jour de fermeture est soit global, et concerne alors l'ensemble du réseau, soit propre à un site déterminé.

Le jour de fermeture porte une date, une raison facultative et un indicateur précisant sa portée. Cette distinction permet de gérer indifféremment un jour férié commun à tous les clubs et une fermeture exceptionnelle d'un seul site.

RM-07 — Création d'une fermeture globale réservée à l'administrateur GLOBAL

Seul un administrateur GLOBAL peut créer une fermeture globale.

Lorsque la demande de création porte sur une fermeture globale, le système vérifie que l'administrateur connecté est de rôle GLOBAL ; dans le cas contraire, la création est refusée. Cela évite qu'un administrateur de site puisse fermer l'ensemble du réseau.

RM-08 — Site obligatoire pour une fermeture non globale

Une fermeture non globale doit obligatoirement désigner un site, et un administrateur SITE ne peut créer une fermeture que sur son propre site.

Si la fermeture n'est pas globale, le système exige qu'un site soit précisé ; à défaut, la création est refusée. Lorsqu'un site est précisé, le système vérifie le périmètre de l'administrateur : un administrateur SITE ne peut viser que son propre site, un administrateur GLOBAL peut viser n'importe quel site.

RM-09 — Suppression d'une fermeture selon le périmètre

Seul un administrateur GLOBAL peut supprimer une fermeture globale ; un administrateur SITE ne peut supprimer que les fermetures de son site.

À la suppression, le système refuse qu'un administrateur SITE supprime une fermeture globale, et vérifie, pour une fermeture de site, que l'administrateur agit bien dans son périmètre. Les règles de suppression sont ainsi symétriques de celles de création.

Les règles de gestion des jours de fermeture sont centralisées dans les services métier afin de garantir un comportement identique pour toutes les opérations. Avant toute création, modification ou suppression, le système vérifie automatiquement le rôle de l'administrateur connecté ainsi que son périmètre d'action. Cette validation empêche un administrateur SITE d'effectuer des opérations en dehors de son propre site et réserve les fermetures globales aux administrateurs GLOBAL.

RM-10 — Visibilité des fermetures selon le rôle

Un administrateur GLOBAL voit toutes les fermetures ; un administrateur SITE voit les fermetures globales et celles de son propre site.

Lors de la consultation de la liste des fermetures, le système filtre les résultats selon l'administrateur connecté : un administrateur GLOBAL obtient l'ensemble, tandis qu'un administrateur SITE obtient les fermetures globales (qui le concernent) et celles rattachées à son site, mais pas celles des autres sites.

3.3. Tarification d'un match

RM-11 — Prix total fixe d'un match

Le prix total d'un match est fixé à 60 €.

Ce montant constant est appliqué à la création de chaque match. Il représente le coût du créneau, indépendamment du terrain, et sert de base au calcul de la part par joueur.

RM-12 — Partage du prix en quatre parts égales

Le prix total est divisé à parts égales entre les quatre joueurs, soit 15 € par joueur.

La part individuelle est calculée automatiquement à la création du match (prix total divisé par le nombre maximal de joueurs). Comme ce maximum est de quatre, la part est toujours de 15 €. Ce calcul centralisé garantit que chaque joueur paie le même montant et que la somme des parts couvre exactement le prix du match.

3.4. Fenêtre de réservation

La règle de fenêtre de réservation est une limite maximale d'anticipation, et non un minimum : elle borne jusqu'à combien de jours à l'avance un membre peut réserver, l'horizon dépendant du profil.

RM-13 — Horizon maximal pour un membre GLOBAL

Un membre GLOBAL peut créer un match jusqu'à 21 jours à l'avance.

Le système calcule le nombre de jours entre la date du jour et la date du match et le compare à la limite de 21 jours pour un membre GLOBAL. Au-delà, la création est refusée. C'est l'horizon le plus large, qui privilégie ce profil sur les créneaux lointains.

RM-14 — Horizon maximal pour un membre SITE

Un membre SITE peut créer un match jusqu'à 14 jours à l'avance.

Pour un membre SITE, la limite d'anticipation est de 14 jours. Cet horizon intermédiaire reflète un accès prioritaire moindre que celui du profil GLOBAL.

RM-15 — Horizon maximal pour un membre LIBRE

Un membre LIBRE peut créer un match jusqu'à 5 jours à l'avance.

Le profil LIBRE dispose de l'horizon le plus court, 5 jours. Il ne peut donc réserver qu'à courte échéance, laissant les créneaux lointains aux profils prioritaires.

RM-16 — Refus des dates passées

Une date de match située dans le passé est systématiquement refusée.

Avant même de vérifier l'horizon maximal, le système rejette toute date antérieure à la date du jour, ce qui empêche la création de matchs incohérents et protège les calculs ultérieurs.

Cette famille de règles s'applique à la création d'un match. Rejoindre un match déjà ouvert par un autre membre n'est pas soumis à cette limite d'horizon, ce point n'étant pas prévu par l'énoncé du projet.

Les règles de fenêtre de réservation sont appliquées exclusivement par le service métier avant la création d'un match. Le système calcule automatiquement l'écart entre la date courante et la date demandée, puis compare cette valeur avec l'horizon maximal autorisé selon le profil du membre (GLOBAL, SITE ou LIBRE). Cette validation garantit que les règles d'anticipation sont identiques pour tous les utilisateurs et qu'elles ne peuvent pas être contournées depuis le frontend.

L'interface Angular effectue des contrôles de saisie afin d'améliorer l'expérience utilisateur, mais la validation finale est toujours réalisée par le backend. Ainsi, même si une requête est envoyée manuellement, les limites de 21 jours, 14 jours, 5 jours ainsi que l'interdiction des dates passées restent appliquées par le serveur. Cette logique est implémentée dans la méthode `validateBookingDelay()` du service de gestion des matchs.

3.5. Disponibilité d'un créneau

RM-17 — Calcul automatique de l'heure de fin

L'heure de fin d'un match est calculée automatiquement à partir de l'heure de début et de la durée standard du site.

Le membre ne saisit que l'heure de début ; le système ajoute la durée de match configurée pour le site afin de déterminer l'heure de fin. Cela garantit une durée homogène pour tous les matchs d'un même site et fournit l'intervalle utilisé par les contrôles de chevauchement et d'horaires.

RM-18 — Absence de chevauchement sur un même terrain

Deux matchs ne peuvent pas occuper le même terrain sur des créneaux qui se chevauchent.

Lors de la création ou de la modification, le système recherche les matchs du terrain dont l'intervalle recoupe celui demandé, en ignorant les matchs annulés ; s'il en trouve, le créneau est déclaré indisponible. Lors d'une modification, le match en cours d'édition est exclu de cette recherche pour ne pas entrer en conflit avec lui-même.

RM-19 — Respect de la pause entre deux matchs

Une durée de pause configurée pour le site doit séparer la fin d'un match du début du suivant sur un même terrain.

La recherche de chevauchement n'utilise pas l'intervalle brut : elle l'élargit de la durée de pause du site avant l'heure de début et après l'heure de fin. Un match qui débiterait exactement à l'heure de fin du précédent est donc refusé tant que la pause n'est pas écoulée, ce qui ménage le temps de remise en état du terrain.

Les règles de disponibilité d'un créneau sont entièrement appliquées par le service métier lors de la création ou de la modification d'un match. Le système calcule automatiquement l'heure de fin à partir de l'heure de début et de la durée configurée pour le site, puis vérifie l'absence de chevauchement avec les autres matchs et le respect de la durée minimale de pause entre deux occupations successives d'un même terrain. Ces contrôles garantissent une planification cohérente et évitent tout conflit de réservation.

3.6. Ouverture et fermeture des sites

RM-20 — Match dans les heures d'ouverture

Un match doit se dérouler intégralement entre l'heure d'ouverture et l'heure de fermeture du site.

Le système refuse un match dont l'heure de début précède l'ouverture, ou dont l'heure de fin dépasse la fermeture. Le contrôle porte sur la totalité du créneau, ce qui empêche un match de se terminer après la fermeture du site.

RM-21 — Blocage par une fermeture globale

Aucun match ne peut être créé, sur quelque site que ce soit, à une date faisant l'objet d'une fermeture globale.

Avant d'accepter un match, le système vérifie l'absence de fermeture globale à la date

demandée. Une fermeture globale s'impose à l'ensemble des sites du réseau.

RM-22 — Blocage par une fermeture de site

Aucun match ne peut être créé à une date faisant l'objet d'une fermeture propre au site du terrain.

Le système vérifie également l'absence de fermeture spécifique au site concerné à la date demandée. Cette vérification, combinée à la précédente, couvre les deux types de fermeture, et s'applique aussi bien à la création qu'à la modification d'un match.

Dans l'implémentation actuelle, ces contrôles sont réalisés avant l'enregistrement du match. Le service métier vérifie successivement les horaires d'ouverture du site, l'absence de fermeture globale, puis l'absence de fermeture spécifique au site concerné. Si l'une de ces validations échoue, une exception métier est levée et la création ou la modification du match est interrompue. Cette logique garantit qu'aucun match ne puisse être planifié en dehors des conditions d'exploitation du site. Les horaires d'ouverture sont chargés par le frontend afin d'améliorer l'expérience utilisateur lors de la saisie, mais la validation définitive reste effectuée par le backend. Cette double vérification garantit que les règles métier restent applicables même si une requête est envoyée en dehors de l'interface Angular.

3.7. Périmètre des membres SITE

RM-23 — Création de match limitée au site de rattachement

Un membre de profil SITE ne peut créer un match que sur un terrain appartenant à son propre site.

À la création, si l'organisateur est de profil SITE, le système vérifie qu'il est rattaché à un site et que ce site correspond à celui du terrain choisi ; sinon, la création est refusée. Cette règle traduit le caractère local de l'abonnement SITE.

Les restrictions appliquées aux membres de profil SITE sont vérifiées par le service métier avant toute création de match ou réservation. Le système compare automatiquement le site de rattachement du membre avec celui du terrain concerné. Si les deux sites ne correspondent pas, l'opération est refusée afin de garantir le respect du périmètre associé à l'abonnement SITE.

RM-24 — Réservation limitée au site de rattachement

Un membre de profil SITE ne peut réserver une place que sur un match organisé sur son propre site.

Lorsqu'un membre SITE rejoint un match, le système compare le site du membre au site du terrain du match ; s'ils diffèrent, la réservation est refusée. Les membres GLOBAL et LIBRE ne sont pas concernés par cette restriction.

3.8. Capacité et statut d'un match**RM-25 — Quatre joueurs au maximum**

Un match accueille au maximum quatre joueurs.

Le nombre maximal est une constante du système ; il borne le nombre de joueurs confirmés et sert de base au calcul de la part par joueur. Toute tentative d'ajouter un cinquième joueur est rejetée.

RM-26 — Statut initial et nombre de joueurs à la création

À sa création, un match est au statut « planifié » et son nombre de joueurs confirmés vaut zéro.

Le nombre de joueurs ne compte que les places effectivement confirmées par paiement. Il démarre donc à zéro, même si l'organisateur dispose dès la création d'une réservation en attente (voir RM-32).

RM-27 — Passage automatique au statut « complet »

Un match passe automatiquement au statut « complet » dès qu'il atteint quatre joueurs confirmés. À chaque confirmation, le nombre de joueurs est incrémenté ; lorsqu'il atteint quatre, le statut bascule de « planifié » à « complet ». Un match complet n'accepte plus de réservation.

RM-28 — Retour au statut « planifié » après désistement

Si un joueur confirmé se désiste et que le nombre repasse sous quatre, le match redevient « planifié ».

L'annulation d'une réservation confirmée décrémente le nombre de joueurs ; si le match était complet, il repasse à « planifié » et redevient disponible. Le statut reflète ainsi en permanence la disponibilité réelle du match.

RM-29 — Modification réservée à l'organisateur

Seul l'organisateur d'un match peut le modifier, et un match annulé ne peut plus être modifié.

La modification vérifie que le demandeur est l'organisateur du match et que le match n'est pas annulé ; sinon, elle est refusée. Les mêmes contrôles de fermeture, d'horaires et de disponibilité du créneau que lors de la création sont alors rejoués.

es règles de capacité et de statut sont appliquées automatiquement par le service métier tout au long du cycle de vie d'un match. Chaque confirmation de réservation met à jour le nombre de joueurs confirmés et peut entraîner un changement automatique du statut du match. De la même manière, une annulation d'une réservation confirmée peut faire repasser un match de l'état « complet » à l'état « planifié ». Le statut du match reflète ainsi en permanence son état réel et le nombre de places effectivement disponibles.

RM-30 — Annulation réservée à l'organisateur

Seul l'organisateur peut annuler son match, et un match déjà annulé ne peut pas l'être une seconde fois.

L'annulation vérifie l'identité de l'organisateur et refuse d'agir sur un match déjà annulé, ce qui évite les états incohérents.

3.9. Réservation

RM-31 — Réservation impossible sur un match complet ou annulé

Aucune réservation n'est acceptée si le match est complet ou annulé.

Le statut du match est contrôlé avant toute réservation : un match complet n'a plus de place, un match annulé n'existe plus fonctionnellement. Ces deux cas bloquent la réservation avec un message explicite.

RM-32 — Réservation et paiement automatiques de l'organisateur

À la création d'un match, l'organisateur se voit attribuer automatiquement une réservation en attente et le paiement en attente correspondant.

L'organisateur occupe une place au même titre que les autres joueurs : il devra payer sa part pour que sa place soit confirmée. Cette réservation et ce paiement sont créés dans la foulée de l'enregistrement du match.

RM-33 — Inscription unique par membre

Un membre ne peut pas réserver deux fois le même match.

Avant d'enregistrer une réservation, le système vérifie qu'il n'existe pas déjà une réservation reliant ce membre à ce match, ce qui empêche un même joueur d'occuper plusieurs places.

RM-34 — Blocage en cas de pénalité active

Un membre sous pénalité active ne peut pas réserver.

La réservation est refusée si le membre a une pénalité dont la date de fin n'est pas encore atteinte. Cette règle prolonge l'effet dissuasif de la pénalité au-delà de la seule création de match.

RM-35 — Blocage en cas de solde impayé

Un membre ayant un solde impayé ne peut pas réserver.

La réservation est refusée si le solde du membre est strictement positif, ce qui l'incite à régler ses dettes avant de réserver de nouveau.

RM-36 — Match privé : seul l'organisateur invite

Dans un match privé, seul l'organisateur peut inscrire d'autres joueurs.

Pour un match privé, le système vérifie que le demandeur de la réservation est l'organisateur ; un tiers ne peut pas s'y inscrire de lui-même. Cela traduit la nature « sur invitation » d'un match privé.

RM-37 — Match public : réservation pour soi-même uniquement

Dans un match public, un membre ne peut réserver que pour lui-même.

Pour un match public, le système vérifie que le membre réservé est bien celui qui fait la demande ; on ne peut pas inscrire un autre membre à sa place. Chaque joueur s'engage ainsi personnellement.

RM-38 — Génération automatique du paiement de la réservation

Toute réservation crée automatiquement un paiement en attente du montant de la part par joueur. Dès qu'une réservation est enregistrée, le paiement associé est créé au statut « en attente » avec le montant de la part. Aucune place ne peut donc exister sans paiement correspondant à honorer.

Lorsqu'une réservation est validée, le backend crée automatiquement le paiement associé au statut « en attente ». Le frontend Angular récupère ensuite les nouvelles informations au travers des services API afin de mettre à jour l'affichage des réservations, des paiements et des places disponibles. Cette organisation garantit une parfaite synchronisation entre l'interface utilisateur et les données enregistrées dans la base de données.

3.10. Paiement et solde

RM-39 — Paiement réservé au membre concerné

Seul le membre titulaire d'une réservation peut en régler le paiement.

Le système vérifie que le membre qui paie est celui de la réservation ; un autre membre ne peut pas payer à sa place. Cela protège l'imputation des paiements.

RM-40 — Refus de payer une réservation annulée

Le paiement d'une réservation annulée est refusé.

Une réservation au statut « annulée » ne peut plus être payée, puisque la place a été libérée. Le système écarte ce cas avant tout traitement de paiement.

RM-41 — États de paiement non rejouables

Un paiement déjà effectué ou annulé ne peut pas être réglé de nouveau ; seule une réservation en attente peut être payée.

Le système refuse un paiement déjà au statut « payé » ou « annulé », et exige que la réservation soit en attente. Cela évite les doubles paiements et les régularisations sur des réservations caduques.

RM-42 — Confirmation de la place au paiement (premier payé, premier servi)

La place n'est définitivement acquise qu'au paiement, et la dernière place revient au premier membre qui paie.

Au moment du paiement, le système recharge l'état du match sous verrou afin de traiter les paiements concurrents de manière strictement ordonnée, puis vérifie que le match n'a pas déjà atteint quatre joueurs. Si la dernière place vient d'être prise par un autre membre ayant payé en premier, le paiement est refusé. Une simple réservation ne bloque donc pas une place : c'est le paiement qui fait foi.

RM-43 — Protection contre les paiements concurrents

L'attribution de la dernière place est protégée par un verrou afin de ne jamais dépasser quatre joueurs.

Le rechargement du match sous verrou au moment du paiement sérialise les paiements simultanés sur un même match. Cela empêche que deux membres voient tous deux la même dernière place comme disponible et la prennent en même temps.

RM-44 — Imputation du solde dû lors du paiement

Si le membre a un solde dû antérieur, ce solde est ajouté au montant réglé puis remis à zéro.

Lorsqu'un membre paie, le système vérifie s'il a un solde positif ; le cas échéant, ce solde est ajouté au montant du paiement en cours, puis remis à zéro. Régler une nouvelle réservation permet ainsi de solder ses dettes et débloque l'accès aux

futures réservations.

RM-45 — Confirmation de la réservation après paiement

Le paiement confirme la réservation, ce qui incrémente le nombre de joueurs du match.

Une fois le paiement enregistré (statut « payé » et date renseignée), la réservation passe au statut « confirmée » et le nombre de joueurs est incrémenté, avec passage éventuel du match au statut « complet ». Le paiement est donc l'événement qui matérialise réellement la participation.

RM-46 — Solde impayé dès qu'il est positif

Un membre est considéré en situation d'impayé dès que son solde est strictement supérieur à zéro.

Le solde sert d'indicateur de dette : toute valeur positive bloque la création de match et la réservation. Il est initialisé à zéro à la création du membre et n'augmente que par le report des parts impayées (voir RM-50).

Les règles de paiement sont centralisées dans le service métier afin de garantir la cohérence des transactions. Avant de confirmer un paiement, le système vérifie que le membre est bien le propriétaire de la réservation, que celle-ci est toujours payable, que le solde du membre autorise l'opération et que la place est encore disponible. Toutes ces validations sont réalisées avant la mise à jour de la réservation, du paiement et du match.

3.11. Annulation

RM-47 — Annulation d'une réservation en attente sans décrétement

L'annulation d'une réservation seulement en attente ne modifie pas le nombre de joueurs du match.

Comme la place n'avait jamais été confirmée ni comptabilisée, son annulation ne décrémente pas le nombre de joueurs. Cela évite de fausser le décompte en retirant une place qui n'avait pas été comptée.

RM-48 — Annulation d'une réservation confirmée avec décrétement

L'annulation d'une réservation confirmée décrémente le nombre de joueurs du match.

Le système mémorise que la réservation était confirmée avant de l'annuler, puis diminue le nombre de joueurs, ce qui peut faire repasser le match de « complet » à « planifié » et libérer la place.

RM-49 — Mise à jour du paiement à l'annulation

À l'annulation d'une réservation, un paiement déjà effectué passe à « remboursé » et un paiement en attente passe à « annulé ».

Le paiement associé est mis à jour selon son état, de sorte que son statut reflète toujours la réalité financière de la réservation après annulation. Une réservation déjà annulée ne peut pas l'être une nouvelle fois.

L'annulation d'une réservation est entièrement gérée par le service métier afin de garantir la cohérence des données. Selon que la réservation était simplement en attente ou déjà confirmée, le système adapte automatiquement les traitements à effectuer : mise à jour éventuelle du nombre de joueurs, changement du statut du match, modification de l'état du paiement et enregistrement de toutes les opérations dans une seule transaction.

3.12. Traitements automatiques et pénalités

RM-50 — Report des parts impayées sur l'organisateur

La veille du match, toute réservation restée impayée est annulée et le montant de sa part est reporté sur le solde de l'organisateur du match.

Un traitement quotidien parcourt les réservations encore en attente de paiement pour les matchs du lendemain, les annule, et ajoute la part correspondante au solde de l'organisateur. L'organisateur assume ainsi financièrement les places non honorées, ce qui le responsabilise sur la composition de son match.

RM-51 — Conversion automatique des matchs privés incomplets

Un match privé prévu le lendemain, encore planifié et comptant moins de quatre joueurs, est automatiquement converti en match public.

Le traitement de minuit identifie ces matchs et les rend publics, en enregistrant la date de conversion. L'ouverture du match permet à d'autres joueurs de compléter une partie qui risquait sinon de ne pas avoir lieu, et optimise l'occupation des terrains.

RM-52 — Pénalité d'une semaine à l'organisateur converti

La conversion automatique d'un match privé applique à son organisateur une pénalité d'une semaine.

Lors de la conversion, le système crée une pénalité dont la date de fin est fixée une semaine plus tard, avec un motif explicite. La pénalité sanctionne le fait de ne pas avoir réuni son groupe à temps.

RM-53 — Match déjà public non reconvertible

Un match déjà public ne peut pas être converti une nouvelle fois.

L'opération de conversion refuse d'agir sur un match déjà public, ce qui évite d'appliquer une pénalité induite ou de réécrire une date de conversion déjà fixée.

RM-54 — Pénalité active bloquante

Une pénalité est active tant que sa date de fin n'est pas atteinte, et une pénalité active bloque la création de match et la réservation.

Le système considère qu'un membre a une pénalité active s'il existe pour lui une pénalité dont la date de fin est postérieure à la date du jour. Cette condition est vérifiée à la création de match et à la réservation.

RM-55 — Exécution quotidienne des traitements

Les deux traitements automatiques s'exécutent chaque jour, le planificateur étant activé.

La conversion des matchs privés s'exécute à minuit et le traitement des impayés peu après. L'activation du planificateur est indispensable pour que ces tâches s'exécutent réellement de façon périodique.

3.13. Authentification et sécurité

RM-56 — Authentification administrateur par e-mail et mot de passe

L'administrateur s'authentifie par son adresse e-mail et son mot de passe.

Le système recherche l'administrateur par e-mail puis compare le mot de passe fourni au hachage stocké. En cas de succès, un jeton est émis avec l'identité et le rôle de l'administrateur.

RM-57 — Stockage du mot de passe en haché

Le mot de passe administrateur est stocké sous forme de hachage BCrypt et n'est jamais conservé en clair.

Le hachage protège les mots de passe en cas de fuite de la base. La vérification se fait en comparant le hachage du mot de passe saisi à celui stocké.

RM-58 — Message d'erreur d'authentification neutre

En cas d'échec d'authentification, un message neutre est renvoyé sans préciser si l'e-mail ou le mot de passe est en cause.

Cette neutralité évite de révéler l'existence d'un compte ou de distinguer un e-mail inconnu d'un mot de passe erroné, ce qui réduit la surface d'attaque.

RM-59 — Authentification membre par matricule

Le membre s'authentifie au moyen de son matricule.

La route de connexion membre permet d'obtenir une session sur la base du matricule. La consultation par matricule, elle, ne délivre pas de jeton : seule la route de connexion authentifie.

RM-60 — Séparation technique des rôles membre et administrateur

Le jeton porte une autorité technique distincte, `ROLE_MEMBER` ou `ROLE_ADMIN`, qui sépare les deux univers.

Les routes de création de match, de réservation et de paiement exigent `ROLE_MEMBER` ; les routes d'administration exigent `ROLE_ADMIN`. Un membre ne peut donc pas atteindre une route d'administration, et inversement, indépendamment du libellé métier `GLOBAL/SITE`.

RM-61 — Routes protégées par défaut

Toute route non explicitement déclarée publique exige au minimum un rôle administrateur.

La configuration de sécurité ouvre nommément les routes publiques (connexion, documentation, consultation des sites et terrains, liste des matchs publics) ; toutes les autres sont protégées, ce qui évite d'exposer une route par omission.

RM-62 — API sans état

L'API est sans état : aucune session n'est conservée côté serveur et chaque requête protégée doit présenter son jeton.

Un filtre valide le jeton à chaque requête. L'absence de session serveur facilite la montée en charge et impose au client de transmettre systématiquement son jeton.

Dans l'architecture actuelle, l'authentification repose sur un mécanisme JWT sans état. Après une connexion réussie, le backend génère un jeton contenant l'identité de l'utilisateur ainsi que son autorité technique (ROLE_ADMIN ou ROLE_MEMBER). Ce jeton est ensuite transmis au frontend, qui le conserve localement et le joint automatiquement aux requêtes protégées grâce à un intercepteur HTTP. Cette approche évite le maintien d'une session côté serveur et facilite la montée en charge de l'application.

3.14. Périmètre et propriété des ressources**RM-63 — Création de membre réservée à l'administration**

La création d'un membre nécessite un rôle administrateur ; il n'existe pas d'inscription publique.

Créer un membre est une action d'administration ; seule la connexion membre par matricule reste publique. Cette restriction empêche un tiers de créer des comptes à volonté via l'API.

RM-64 — Périmètre de l'administrateur SITE sur les sites et terrains

Un administrateur SITE ne peut agir que sur les sites et terrains de son périmètre ; un administrateur GLOBAL a un accès complet.

Pour chaque opération sensible sur un site ou un terrain, un service d'autorisation vérifie le périmètre : un administrateur GLOBAL passe sans restriction, un administrateur SITE est refusé dès que la ressource relève d'un autre site. Ce contrôle est réalisé côté serveur.

RM-65 — Périmètre de l'administrateur SITE sur les membres

Un administrateur SITE ne peut créer, modifier ou gérer que les membres rattachés à son propre site.

Lors de la création, de la modification ou de la suppression d'un membre, le système vérifie que le membre concerné relève du site de l'administrateur SITE ; sinon, l'opération est refusée. La liste des membres renvoyée à un administrateur SITE est également filtrée sur son site.

RM-66 — Périmètre de l'administrateur SITE sur les matchs

Un administrateur SITE ne voit et ne gère que les matchs de son site ; un administrateur GLOBAL voit l'ensemble.

La liste des matchs renvoyée à un administrateur est filtrée selon son périmètre, et les opérations d'administration sur un match (telles que la conversion) vérifient que le match relève bien du site de l'administrateur SITE.

RM-67 — Accès aux données d'un membre restreint à son titulaire ou à un administrateur

La consultation de la fiche, du solde ou de la pénalité d'un membre est limitée au membre concerné ou à un administrateur.

Le système vérifie que le demandeur est soit le membre lui-même, soit un administrateur, avant de renvoyer ces informations, ce qui protège les données personnelles et financières de chaque membre.

RM-68 — Propriété des réservations

Un membre ne peut consulter ou annuler que ses propres réservations.

Les opérations de consultation par membre et d'annulation vérifient que la réservation appartient bien au membre connecté ; à défaut, l'accès est refusé. Un membre ne peut donc pas agir sur la réservation d'un autre.

RM-69 — Propriété des paiements

Un membre ne peut consulter et régler que les paiements liés à ses propres réservations.

Les opérations de paiement et de consultation des paiements vérifient que le paiement concerne bien le membre connecté ; un membre ne peut pas accéder au paiement d'autrui.

Les règles de périmètre et de propriété des ressources sont appliquées systématiquement avant toute opération sensible. Le backend vérifie l'identité de l'utilisateur authentifié ainsi que ses autorisations avant d'autoriser l'accès ou la modification d'une ressource. Cette validation permet de garantir qu'un utilisateur ne puisse consulter ou modifier que les données auxquelles il est effectivement autorisé à accéder. Les règles de périmètre et de propriété des ressources sont appliquées systématiquement avant toute opération sensible. Le backend vérifie l'identité de l'utilisateur authentifié ainsi que ses autorisations avant d'autoriser l'accès ou la modification d'une ressource. Cette validation permet de garantir qu'un utilisateur ne puisse consulter ou modifier que les données auxquelles il est effectivement autorisé à accéder.

3.15. Validation des saisies**RM-70 — Validation des données entrantes**

Les données reçues par l'API sont validées avant tout traitement métier.

Les objets de requête portent des contraintes (champs obligatoires, format d'e-mail, etc.) vérifiées automatiquement à l'entrée des contrôleurs. Une requête mal formée est rejetée immédiatement, avec un message en français, sans atteindre la couche métier.

RM-71 — Format du matricule selon le profil

Le matricule doit respecter le format du profil : G suivi de 4 chiffres pour un membre GLOBAL, S suivi de 5 chiffres pour un membre SITE, L suivi de 5 chiffres pour un membre LIBRE.

À la création d'un membre, le format est contrôlé selon le profil ; un matricule non conforme est refusé. Le préfixe véhicule ainsi de façon fiable l'information de profil exploitée par les règles de réservation.

RM-72 — Unicité du matricule

Le matricule d'un membre est unique dans le système.

La création échoue si le matricule est déjà utilisé. Cette unicité est nécessaire à l'authentification par matricule et à l'imputation correcte des réservations et paiements.

RM-73 — Unicité de l'adresse e-mail

L'adresse e-mail d'un membre est unique dans le système.

La création échoue si l'e-mail est déjà utilisé, ce qui évite les doublons de compte et les ambiguïtés d'identité.

RM-74 — Rattachement obligatoire d'un membre SITE

Un membre de profil SITE doit être rattaché à un site existant.

La création d'un membre SITE sans site est refusée. Ce rattachement est indispensable aux règles de périmètre, qui comparent le site du membre à celui du match.

RM-75 — Initialisation du solde à la création

Le solde d'un nouveau membre est initialisé à zéro.

Un membre commence sans dette. Son solde n'augmente qu'avec le report des parts impayées et se réduit lors d'un paiement intégrant ce solde.

Les validations de saisie sont réalisées à deux niveaux complémentaires. Le frontend Angular contrôle les champs obligatoires et le format des données afin d'améliorer l'expérience utilisateur, tandis que le backend effectue la validation définitive avant tout traitement métier. Une donnée invalide est immédiatement rejetée et aucune opération n'est exécutée sur la base de données.

ID	Description	Entités	Service Spring	Contrôleur	Repository	Composant Angular	Service Angular	Scheduler
R MO 1	Un créneau dure 1h30 avec 15 minutes entre deux matchs.	Site, Terrain, Match	MatchServiceImpl.create Match()	MatchController	MatchRepository	member- create- match.page e.ts	match- es- api.service.ts	
R MO 2	Les horaires sont propres à chaque site et valables pour une année civile.	Site	MatchServiceImpl.validateSiteOpeningHours()	SiteController/MatchController	SiteRepository	admin- sites.page. ts	sites- api.service.ts	
R MO 3	Chaque site peut avoir ses propres jours de fermeture.	JourFermeture, Site	validateSiteNotClosed()	SiteController	JourFermetureRepository	admin- jours- fermeture. page.ts	jours- fermeture- api.service.ts	

R	Des jours	JourFerme		SiteController/Match C	JourFermetureRe	admin-	jours-	
---	-----------	-----------	--	---------------------------	-----------------	--------	--------	--

MO 4	de fermeture globaux peuvent bloquer tous les sites.	ture	validateSiteNotClosed() ()	ontroller	pository	jours- fermeture. page.ts	ferme t ure- api.ser vice.ts	
R MO 5	Un match peut être public ou privé.	Match, TypeMatc h	createMatch()	MatchController	MatchRepository	member- create- match.pag e.ts	match es- api.ser vice.ts	
R MO 6	Un match privé nécessite 4 joueurs.	Match, Reservatio n	incrementPlayers(), create()	ReservationController	ReservationRepo sitory	member- create- match.pag e.ts	reserv ations - api.ser vice.ts	MatchS c hedul er
R MO 7	Si un match privé n'est pas complet la veille, il devient public.	Match	checkAndConvertExpired PrivateMatches(), convertToPublic()	MatchController	MatchRepository	member- public- matches.p age.ts	match es- api.ser vice.ts	MatchS c hedul er
R MO 8	Si le match privé reste incomplet , le	Penalite, Membre, Match	addPenalty()	MembreController/M at chController	PenaliteReposito r y	member- profile.pag e.ts	memb r es- api.ser vice.ts	MatchS c hedul er

	responsable reçoit une pénalité d'une semaine.							
RM09	Dans un match privé, le responsable ajoute les autres joueurs.	Match, Reservation, Membre	ReservationServiceImpl.create()	ReservationController	ReservationRepository	member-create-match.page.ts	reservations-api.service.ts	
RM10	Quand le match devient public, il est visible par les membres autorisés.	Match	getPublicAvailableMatches()	MatchController	MatchRepository	member-public-matches.page.ts	matches-api.service.ts	Match
RM11	Le prix total d'un match est de 60 euros.	Match	createMatch()	MatchController	MatchRepository	member-create-match.page.ts	matches-api.service.ts	
RM12	Le prix est réparti entre 4 joueurs, soit 15 euros par joueur.	Match, Paiement	createMatch(), pay()	PaiementController	PaiementRepository	member-payment.page.ts	payments-api.service.ts	
RM13	Si un joueur n'a pas payé la veille, sa place redevient disponible.	Reservation, Paiement	checkUnpaidBeforeMatch()	PaiementController	PaiementRepository, ReservationRepository	member-reservations.page.ts	reservations-api.service.ts	Scalpa
RM14	Dans un match public, la validation se fait uniquement après paiement.	Reservation, Paiement	pay(), confirm()	PaiementController	PaiementRepository	member-payment.page.ts	payments-api.service.ts	

R M1 5	Premier payé = premier servi.	Match, Reservati o n	create(), pay(), incrementPlayers()	ReservationController / PaiementController	ReservationRepo sitory	member -public- matches. page.ts	reser v ations - api.se r vice.t s	
-----------	----------------------------------	----------------------------	--	---	---------------------------	---	---	--

R M1 6	Dans un match public, l'organisateur ne peut pas réserver pour un autre joueur.	Match, Réservation	ReservationServiceImpl. create()	ReservationController	ReservationRepository	member-public-matches. page.ts	reservations- api.service.ts	
R M1 7	Si un match public n'est pas complet, l'organisateur paie le solde.	Membre, Match, Paiement	checkUnpaidBeforeMatch()	PaiementController	PaiementRepository	member-payment. page.ts	payments-api.service.ts	Scalpa
R M1 8	Un organisateur avec	Membre	hasOutstandingBalance()	MembreController/MatchController	MembreRepository	member-create-match. page.ts	members-api.service.ts	

	solde impayé ne peut pas créer de réservation.					e.ts	vice.ts	
R M1 9	Lors d'un futur paiement, le solde dû est ajouté automatiquement.	Membre, Paiement	PaiementServiceImpl. pay()	PaiementController	PaiementRepository	member-payment. page.ts	payments-api.service.ts	
R M2 0	Membre global Gxxxx : réservation 21 jours avant sur tous les sites.	Membre, TypeMembre	validateMatricule(), validateBookingDelay()	MembreController/MatchController	MembreRepository	admin-members. page.ts	members-api.service.ts	
R M2 1	Membre site Sxxxx : réservation 14 jours avant uniquement sur son site.	Membre, Site	validateBookingDelay(), ReservationServiceImpl. create()	ReservationController	MembreRepository	member-public-matches. page.ts	reservations-api.service.ts	

RM2.2	Membre libre Lxxxxx : réservation 5 jours avant sur tous les sites.	Membre	validateBookingDelay()	MatchController	MembreRepository	member-create-match.page.ts	matches-api.service.ts	
-------	--	--------	------------------------	-----------------	------------------	-----------------------------	------------------------	--

R M2 3	Tous les participants doivent appartenir à une catégorie valide.	Membre, TypeMembre	validateMatricule()	MembreController	MembreRepository	admin-members.page.ts	members-api.services	
R M2 4	Un membre site est visible dans tous les sites mais ne réserve que dans son site.	Membre, Site	ReservationServiceImpl.create()	ReservationController	ReservationRepository	member-public-matches.page.ts	reservations-api.services	
R M2 5	Deux types d'administrateurs existent : global et site.	Administrateur, TypeAdministrateur	AuthServiceImpl.login()	AuthController	AdministrateurRepository	admin-login.page.ts	auth-api.services	
R M2 6	L'administrateur global peut gérer tous les sites.	Administrateur, Site	services admin	SiteController	SiteRepository	admin-sites.page.ts	sites-api.services	
R M2 7	L'administrateur de site ne	Administrateur, Site	contrôle siteId à renforcer	SiteController/TerrainController	SiteRepository/TerrainRepository	admin-terrains.page.ts	terrain-s-api.service	

Flux Communication frontend/backend

Transaction entre Angular et Spring Boot se fait via HTTP/JSON. Angular utilise ses services API pour appeler les endpoints REST. Le backend répond avec des DTO au format JSON.

Cas	Flux technique complet
Connexion administrateur	admin-login.page.ts -> AuthApiService.loginAdmin() -> AuthController.login() AuthServiceImpl.login() -> AdministrateurRepository -> JwtConfig -> LoginResponse
Connexion membre	page membre -> MemberSessionService.login() -> AuthApiService.loginMembre() MembreController.login() ->

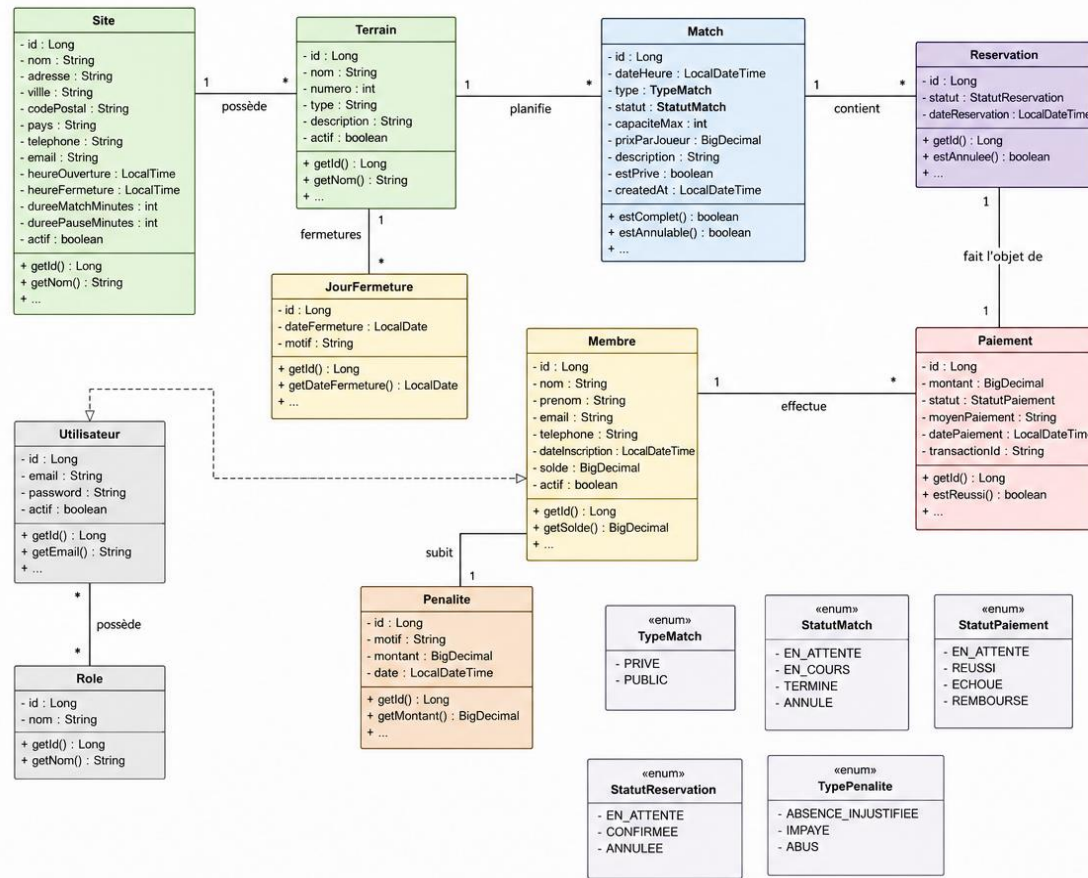
	MembreServiceImpl/getByMatricule() -> session membre
Création d'un match	member-create-match.page.ts -> MatchesApiService.create() -> MatchController.createMa -> MatchServiceImpl.createMatch() -> TerrainService + MembreService + MatchRepository -> PostgreSQL
Réservation d'un match	member-public-matches.page.ts -> ReservationsApiService.create() ReservationController.create() -> ReservationServiceImpl.create() -> MatchRepository + MembreRepository + ReservationRepository -> PostgreSQL
Paielement	member-payments.page.ts -> PaiementsApiService.pay() -> PaiementController.pay() -> PaiementServiceImpl.pay() - > ReservationService.confirm() -> PaiementRepository -> PostgreSQL
Annulation	page réservation/admin -> ReservationsApiService.cancel() ou MatchesApiService.cance Controller -> ServiceImpl -> Repository -> mise à jour statut
Conversion privé vers public	MatchScheduler -> MatchServiceImpl.checkAndConvertExpiredPrivateMatches() -> MatchServiceImpl.convertToPublic() -> MatchRepository -> visibilité côté member-public- matches.page.ts

4. Modèle de données

4. Modèle de données

4.1. Diagramme de classes

Le diagramme ci-dessous représente le modèle de données de l'application *PadelMultiple*.



OÙ PLACER CE DIAGRAMME ?

Ce diagramme de classes doit être placé dans le chapitre 4 - Modèle de données.

Plus précisément :

Chapitre 4 : Modèle de données
Section 4.1 : Diagramme de classes

Emplacement exact conseillé :

→ Juste après l'introduction de la section 4.1, avant la sous-section 4.2 (Entités et attributs).

Ordre recommandé dans le chapitre 4 :

4. Modèle de données
 - 4.1. Diagramme de classes (ce diagramme)
 - 4.2. Entités et attributs
 - 4.3. Relations
 - 4.4. Énumérations de statut

Pourquoi ici ?

- Le diagramme offre une vue d'ensemble avant d'entrer dans le détail des entités et des relations.
- Il aide le lecteur à comprendre la structure de la base de données dès le début du chapitre.

LÉGENDE

- 1 Un
- * Plusieurs
- ◆ Association (relation)
- ▷ Héritage

4.1. Entités et attributs

Le modèle de données s'organise autour du match, qui représente la réservation d'un terrain par un organisateur, et des entités qui gravitent autour de lui. Le tableau ci-dessous récapitule les entités et leurs principaux attributs.

Entité	Attributs principaux
Site	nom, adresse, heure d'ouverture, heure de fermeture, durée de match (min), durée de pause entre matchs (min), année civile
Terrain	nom, prix ; rattaché à un Site
JourFermeture	date, raison (facultative), indicateur global ; rattaché à un Site si fermeture locale

Membre	matricule (unique), nom, prénom, e-mail, profil (GLOBAL/SITE/LIBRE), solde ; Site si profil SITE
Administrateur	matricule, nom, prénom, e-mail (unique), mot de passe haché, rôle (GLOBAL/SITE) ; Site si rôle SITE

Entité	Attributs principaux
Match	dates de début et de fin, type (PRIVÉ/PUBLIC), statut, nombre de joueurs, prix total, prix par joueur, date de conversion éventuelle ; rattaché à un Terrain et à un Membre organisateur
Reservation	statut ; relie un Membre à un Match ; possède un Paiement
Paiement	montant, statut, date de paiement ; rattaché à une Réservation
Penalite	date de fin, motif ; rattachée à un Membre

4.4. Relations

Les relations principales entre entités sont les suivantes :

- un site possède plusieurs terrains et plusieurs jours de fermeture ;
- un terrain appartient à un site et supporte plusieurs matchs ;
- un membre de profil SITE et un administrateur de rôle SITE sont rattachés à un site ;
- un membre organise plusieurs matchs, et possède plusieurs réservations et plusieurs pénalités ;
- un match est lié à un terrain et à un organisateur, et regroupe plusieurs réservations ;
- une réservation relie un membre et un match, et possède un unique paiement.
- Ces relations sont implémentées dans le backend au moyen des annotations JPA (@ManyToOne, @OneToMany et @OneToOne). Elles permettent à Hibernate de gérer automatiquement les liens entre les entités tout en garantissant l'intégrité des données. Les repositories exploitent ces relations pour charger les informations nécessaires aux règles métier sans dupliquer les données.

4.5. Énumérations de statut

Plusieurs entités s'appuient sur des énumérations qui matérialisent leur cycle de vie.

Énumération	Valeurs et signification
Profil de membre	GLOBAL (tous les sites, 21 jours), SITE (son site, 14 jours), LIBRE (tous les sites, 5 jours)

Rôle d'administrateur	GLOBAL (tout le réseau), SITE (son site uniquement)
Type de match	PRIVÉ (sur invitation de l'organisateur), PUBLIC (ouvert à tous)

Statut de match	PLANIFIÉ (créé, non complet), COMPLET (quatre joueurs confirmés), ANNULÉ
Statut de réservation	EN_ATTENTE (place réservée, non payée), CONFIRMÉE (payée), ANNULÉE (place libérée)
Statut de paiement	EN_ATTENTE, PAYÉ, REMBOURSE (annulation d'une réservation payée), ANNULÉ (annulation d'une réservation non payée)

5. Architecture technique

5.1. Organisation en couches

Le backend, développé en Java 21 avec Spring Boot 3.5 sous le package `com.padelPlay`, suit une architecture en couches clairement séparées. Les contrôleurs reçoivent les requêtes HTTP et renvoient les réponses, sans porter de logique métier. Les services (chacun défini par une interface et une implémentation) concentrent l'intégralité des règles métier décrites au chapitre 3. Les repositories, fondés sur Spring Data JPA, assurent l'accès aux données. Les entités représentent le modèle persistant, les objets de requête et de réponse servent aux échanges avec le frontend, et des mappers assurent la conversion entre entités et objets d'échange. Les exceptions métier sont gérées de manière centralisée, et un paquet de configuration regroupe la sécurité, le JWT, le partage de ressources entre origines (CORS) et la documentation. Un paquet dédié contient les tâches planifiées.

Afin de respecter le principe de responsabilité unique (Single Responsibility Principle), chaque couche possède un rôle bien défini. Les contrôleurs assurent uniquement la communication HTTP avec le frontend, les services appliquent les règles métier, les repositories gèrent la persistance des données et les DTO permettent les échanges entre le backend et le frontend sans exposer directement les entités JPA. Cette organisation facilite les tests, la maintenance et l'évolution de l'application.

Deux services transverses encadrent la sécurité applicative : un service d'autorisation des administrateurs, qui vérifie le périmètre GLOBAL ou SITE de l'administrateur courant, et un service de résolution du membre authentifié, qui identifie le membre courant et contrôle qu'une ressource lui appartient bien.

5.2. Sécurité applicative

La sécurité repose sur Spring Security en mode sans état et sur des jetons JWT. Un filtre valide le jeton à chaque requête, et les mots de passe administrateur sont hachés avec BCrypt. Les autorisations sont déclarées explicitement, route par route et méthode HTTP par méthode HTTP, dans la configuration de sécurité : sont distinguées les routes publiques (connexion, documentation, consultation des sites et terrains, liste des matchs publics), les routes réservées aux membres et les routes réservées aux administrateurs. Toute route non explicitement ouverte exige au minimum un rôle administrateur. Le périmètre des administrateurs SITE et l'appartenance des ressources aux membres sont vérifiés en complément dans la couche service.

5.3. Frontend

Le frontend est développé avec Angular. Il consomme exclusivement l'API REST au format JSON, ne contient aucune requête SQL et n'accède jamais directement à la base de données. Il est organisé par fonctionnalité (accueil, espace membre, espace administrateur) et par couche transverse (cœur applicatif, éléments partagés). Il gère séparément les sessions administrateur et membre, attache le jeton approprié à chaque requête au moyen d'un intercepteur, protège ses routes par des gardes, et effectue des validations d'expérience utilisateur (date maximale dans le formulaire de création de match, format des matricules) cohérentes avec les règles serveur, ces dernières restant la référence.

Le frontend est organisé selon une architecture modulaire afin de séparer clairement les responsabilités. Les composants assurent l'affichage et les interactions avec l'utilisateur, les services API communiquent avec le backend au moyen de requêtes HTTP, les services de session gèrent l'authentification locale et les Guards protègent les routes nécessitant une connexion. Cette organisation facilite la maintenance, les tests et l'évolution de l'application.

5.4. Base de données et déploiement

La base de données est PostgreSQL, démarrée via docker-compose et exposée sur le port hôte 5440. Le backend écoute sur le port 8080 et le frontend sur le port 4200, ce dernier étant relié au backend par un proxy de développement. Le schéma de base est géré par l'outil de mapping objet-relationnel, et un jeu de données de démonstration (sites, terrains, jours de fermeture, membres, administrateurs et matchs aux dates relatives à la date courante) est créé au premier démarrage lorsque la base est vide, afin de rendre l'application immédiatement exploitable pour une démonstration.

5.5. Tests automatisés

L'application est couverte par des tests automatisés. Côté backend, des tests unitaires et de tranche (JUnit et Mockito) portent sur les contrôleurs, les services, les mappers et les repositories, y compris les

composants de sécurité ajoutés (autorisation des administrateurs, API des jours de fermeture). Côté frontend, des tests couvrent les pages et services principaux. La suite de tests est destinée à être relancée sur la machine de remise afin de confirmer un état stable avant l'évaluation.

Les tests automatisés ont été conçus pour vérifier les principales couches de l'application de manière indépendante. Les tests des contrôleurs valident les endpoints REST, les codes de réponse HTTP et les échanges avec les services. Les tests des services vérifient les règles métier au moyen d'objets simulés (Mockito), sans dépendre de la base de données. Les tests des repositories contrôlent le comportement des requêtes JPA et les tests des mappers garantissent la cohérence des transformations entre les entités et les DTO.

6. Aperçu de l'API REST

L'API expose les ressources suivantes, préfixées par `/api`. Les niveaux d'accès indiqués correspondent à ce qui est appliqué côté serveur.

Ressource	Opérations et accès
<code>/api/auth</code>	Connexion administrateur (public).
<code>/api/membres</code>	Connexion membre publique ; création, modification, suppression réservées à l'administration ; consultation limitée au membre concerné ou à un administrateur.
<code>/api/sites</code> , <code>/api/terrains</code>	Lecture publique ; écriture réservée à l'administration, dans la limite du périmètre.
<code>/api/jours-fermeture</code>	Gestion réservée à l'administration (fermetures globales ou par site).
<code>/api/matches</code>	Création, modification, annulation par le membre ; liste publique des matchs publics ; conversion en public et vues globales réservées à l'administration.
<code>/api/reservations</code>	Création et annulation par le membre ; consultation limitée au membre concerné ou à un administrateur.
<code>/api/paiements</code>	Réservées au membre concerné.

L'API est documentée via Swagger / OpenAPI, accessible à l'adresse `/swagger-ui.html`.

7. Exploitation

Prérequis : Java 21, Node.js et npm récents, et Docker (optionnel, pour PostgreSQL).

- Backend : exécuter la construction puis le démarrage via le wrapper Maven ; le service écoute sur le port 8080 et la documentation Swagger est disponible sur /swagger-ui.html.
- Frontend : installer les dépendances puis lancer le serveur de développement, accessible sur le port 4200.
- Base de données : démarrer PostgreSQL via docker-compose (port 5440) ; le schéma et les données de démonstration sont créés au premier démarrage si la base est vide.
- Tests : exécuter la suite de tests backend via le wrapper Maven, et la suite frontend via npm.

8. Limites connues et perspectives

- Aucun paiement bancaire réel n'est intégré : le paiement est une opération applicative interne dont seul le statut est suivi.

- Aucune notification (e-mail, SMS, temps réel) n'est envoyée lors des événements métier.
- Le schéma de base est géré par l'outil de mapping plutôt que par des migrations versionnées.
- La règle de fenêtre de réservation s'applique à la création d'un match ; rejoindre un match déjà ouvert n'y est pas soumis, l'énoncé ne le prévoyant pas.
- Perspectives d'évolution : migrations de base versionnées, notifications, paiement réel, statistiques d'exploitation avancées.

9. Conclusion

PadelMultiple répond à l'ensemble des besoins d'un système de réservation multi-sites pour clubs de padel : profils de membres aux droits différenciés, matchs privés et publics, confirmation de la place au paiement selon le principe « premier payé, premier servi », gestion des soldes et des pénalités, automatisation de la conversion des matchs privés incomplets, et administration dotée d'un contrôle d'accès par périmètre. La sécurité repose sur une authentification par jetons et une séparation stricte des rôles, et les saisies sont validées côté serveur.

Les soixante-quinze règles métier détaillées au chapitre 3 ont été établies à partir de l'analyse directe du code dans son état actuel : chacune correspond à un comportement implémenté et vérifiable, ce qui garantit la cohérence entre ce cahier des charges et l'application livrée.

Ce projet met en œuvre les principaux concepts étudiés durant le bachelier en développement d'application : architecture en couches, API REST, Spring Boot, Angular, JWT, JPA/Hibernate, PostgreSQL, validation des données, tests automatisés et séparation des responsabilités. Il constitue une application complète, évolutive et conforme aux objectifs fonctionnels fixés au début du projet.

10. Complément de compatibilité avec le projet livré

Cette partie complète le cahier des charges existant sans supprimer son contenu. Elle sert d'addendum de compatibilité : elle précise les éléments du code qui doivent être cités pendant la défense, corrige les points à nuancer et ajoute les détails techniques manquants pour relier clairement le document au projet Angular + Spring Boot + PostgreSQL livré.

Le projet de référence analysé est le dossier padellAout-master. Le backend réel se trouve sous `src/main/java/com/padell/padell` et le frontend sous `frontend/src/app`. Cette précision est importante car le cahier initial décrit correctement l'architecture générale, mais certains noms de packages et certains détails de tarification doivent être explicitement alignés avec le code.

Point vérifié	État dans le cahier actuel	Complément à retenir pour la compatibilité
Package backend	Le document mentionne <code>com.padelPlay</code> .	Le code livré utilise <code>com.padell.padell</code> . À dire et à montrer dans <code>PadellApplication.java</code> et dans tous les packages Java.
Prix du terrain	Le document indique que le terrain porte un prix utilisé comme base.	L'entité <code>Terrain</code> possède bien un prix, mais <code>MatchServiceImpl</code> applique actuellement une constante <code>MATCH_PRICE = 60.0</code> . La part par joueur vaut $60 / 4 = 15$.
Admin SITE	Le périmètre est décrit globalement.	Le contrôle exact se fait dans <code>AdminAuthorizationService</code> : <code>checkSiteAccess()</code> , <code>checkTerrainAccess()</code> , <code>checkMembreAccess()</code> , <code>filterMembres()</code> , <code>filterMatchDtos()</code> .
Authentification membre	Le principe est décrit.	La connexion membre passe par <code>POST /api/membres/login</code> dans <code>MembreController</code> . Le token est placé dans <code>MembreResponse.token</code> .
Gestion des erreurs	Elle est mentionnée rapidement.	Elle doit être détaillée : <code>BusinessException</code> , <code>ResourceNotFoundException</code> , <code>MatchCreationException</code> , <code>GlobalExceptionHandler</code> et <code>ErrorResponse</code> .
DTO et mappers	Présents mais pas assez localisés.	Les DTO sont dans <code>dto/request</code> et <code>dto/response</code> . Les mappers sont dans <code>mapper/*.java</code> et transforment les entités en réponses JSON.

10.1. Vérification générale de compatibilité

Après comparaison avec les dossiers du projet, le cahier des charges reste compatible dans son objectif métier : il décrit bien une application multi-sites permettant aux membres de créer des matchs, de réserver, de payer, d'annuler et aux administrateurs de gérer les ressources. Les règles de paiement, de pénalité, de transformation des matchs privés en matchs publics et de limitation par profil sont présentes dans le code.

La compatibilité doit cependant être renforcée par des références de code plus précises. Pendant la défense, il ne suffit pas de dire qu'une règle existe : il faut ouvrir le contrôleur qui reçoit la requête, le service qui contient la règle métier, le repository qui interroge la base et, si nécessaire, le mapper ou le DTO qui transporte les données.

- Chaîne à expliquer : composant Angular -> service API Angular -> contrôleur Spring -> service métier -> mapper -> repository JPA -> entité PostgreSQL.
- Les règles métier critiques sont dans les services implémentés : `MatchServiceImpl`, `ReservationServiceImpl`, `PaiementServiceImpl`, `MembreServiceImpl`.
- La sécurité est séparée entre frontend et backend : guards côté Angular, `SecurityConfig` et

JwtAuthenticationFilter côté Spring Boot.

- Les erreurs métier ne sont pas gérées dans les contrôleurs directement : elles sont levées dans les services

puis standardisées par GlobalExceptionHandler.

Architecture applicative vérifiée dans le projet livré

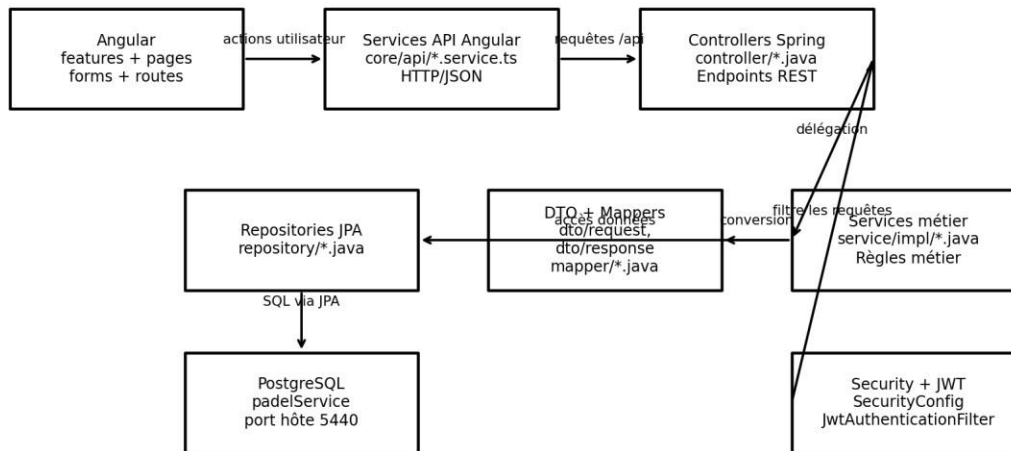


Figure 4 — Architecture réelle du projet et communication entre les couches

11. Architecture backend détaillée à ajouter au cahier

Le backend est une application Spring Boot structurée en couches. Cette organisation permet de séparer la réception HTTP, les règles métier, la persistance, les objets d'échange et la sécurité. Le dossier racine réel est `src/main/java/com/padell/padell`.

Dossier backend	Rôle dans le projet	Exemples à montrer
config	Configuration technique : sécurité, CORS, JWT, Swagger.	SecurityConfig.java, JwtAuthenticationFilter.java, JwtConfig.java, CorsConfig.java, SwaggerConfig.java
controller	Entrée REST de l'application. Les contrôleurs reçoivent les requêtes HTTP et appellent les services.	AuthController.java, MembreController.java, MatchController.java, ReservationController.java, PaiementController.java
dto/request	Objets reçus depuis le frontend. Ils évitent d'exposer directement les entités JPA.	CreateMatchRequest.java, MembreRequest.java, ReservationRequest.java, SiteRequest.java, TerrainRequest.java
dto/response	Objets renvoyés au frontend. Ils contiennent seulement les informations utiles à l'interface.	MatchDto.java, MembreResponse.java, ReservationResponse.java, PaiementResponse.java, LoginResponse.java
entity	Modèle persistant JPA. Chaque classe représente une table ou une structure persistante.	Site.java, Terrain.java, Match.java, Membre.java, Reservation.java, Paiement.java, Penalite.java

entity/enums	Énumérations des statuts et types métier.	TypeMembre.java, TypeMatch.java, StatutMatch.java, StatutReservation.java, StatutPaiement.java
exception	Gestion centralisée des erreurs métier et techniques.	BusinessException.java, ResourceNotFoundException .java, GlobalExceptionHandler.java, ErrorResponse.java
mapper	Conversion entre entités JPA et DTO.	MatchMapper.java, MembreMapper.j ava, ReservationMapper.java, PaiementMapper.java
repository	Accès base de données avec Spring Data JPA.	MatchRepository.java, MembreRepository. java,

		ReservationRepository.java, PaiementRepository.java
scheduler	Tâches automatiques quotidiennes.	MatchScheduler.java
service	Interfaces des services métier.	MatchService.java, ReservationService.java, PaiementService.java, MembreService.java
service/impl	Implémentations où se trouvent les règles métier principales.	MatchServiceImpl.java, ReservationServiceImpl.java, PaiementServiceImpl.java, AdminAuthorizationService.java

11.1. Rôle exact des contrôleurs

Les contrôleurs ne doivent pas être présentés comme la couche qui décide des règles métier. Leur rôle est de recevoir la requête, extraire les paramètres, vérifier éventuellement l'identité courante et déléguer au service. La logique métier est ensuite appliquée dans les services.

Contrôleur	Endpoint principal	Rôle
AuthController	/api/auth/login	Connexion administrateur avec e-mail et mot de passe.
MembreController	/api/membres, /api/membres/login	Gestion des membres et connexion par matricule.
MatchController	/api/matches	Création, consultation, modification, annulation et conversion de matches.
ReservationController	/api/reservations	Création, consultation et annulation des réservations.
PaiementController	/api/paiements	Paiement d'une réservation et consultation des paiements.
SiteController	/api/sites	Gestion des sites.
TerrainController	/api/terrains	Gestion des terrains.
JourFermetureController	/api/jours-fermeture	Gestion des jours de fermeture globaux ou par site.

11.2. Services métier principaux

Les services sont la partie à montrer quand le professeur demande où sont codées les règles métier. Les annotations @Service et @Transactional montrent que ces classes exécutent la logique métier et contrôlent la cohérence des modifications en base.

Service	Responsabilité principale	Règles visibles
MatchServiceImpl	Créer, modifier, annuler, convertir les matchs et gérer le nombre de joueurs.	solde impayé, pénalité active, fenêtre de réservation, horaires, fermeture, chevauchement, pause, prix fixe, réservation automatique de l'organisateur
ReservationServiceImpl	Créer, confirmer et annuler les réservations.	match complet/annulé, inscription unique, match privé, match public, restriction membre SITE, paiement automatique
PaiementServiceImpl	Payer une réservation et gérer les impayés.	propriétaire du paiement, paiement non jouable, verrou pessimiste, solde ajouté au paiement, confirmation au paiement

MembreServiceImpl	Créer et gérer les membres.	format matricule, unicité matricule/e-mail, rattachement SITE, solde initial, pénalité active
AdminAuthorizationService	Limitier l'action des administrateurs selon GLOBAL ou	périmètre site, terrain, membre et match

	SITE.	
CurrentMemberService	Identifier le membre connecté et protéger ses ressources.	un membre ne peut agir que sur ses propres données

11.3. JPA et localisation de la persistance

Le JPA du projet est localisé dans deux familles de dossiers : les entités et les repositories. Les entités portent les annotations @Entity, @Table, @ManyToOne, @OneToMany, @OneToOne, @JoinColumn et @Enumerated. Les repositories héritent de JpaRepository et fournissent les méthodes d'accès aux tables.

Élément JPA	Fichier	Preuve à montrer
Entité Site	entity/Site.java	@Entity, @Table(name = "sites"), relations vers terrains et jours de fermeture.
Entité Terrain	entity/Terrain.java	@ManyToOne vers Site, @OneToMany vers Match.
Entité Match	entity/Match.java	@ManyToOne vers Terrain et Membre organisateur, @OneToMany vers Reservation.
Entité Reservation	entity/Reservation.java	@ManyToOne vers Match et Membre, @OneToOne vers Paiement.
Entité Paiement	entity/Paiement.java	@OneToOne vers Reservation, statut et date de paiement.
Entité Penalite	entity/Penalite.java	@ManyToOne vers Membre, dateFin, motif.
Repository avec verrou	repository/MatchRepository.java	@Lock(LockModeType.PESSIMISTIC_WRITE) sur findByIdForUpdate pour protéger la dernière place.
Repository chevauchement	repository/MatchRepository.java	@Query qui vérifie dateDebut < end et dateFin > start.

11.4. DTO et mappers à expliquer

Un DTO est un objet de transfert de données. Il sert à échanger des données entre le frontend et le backend sans exposer directement l'entité JPA. Dans ce projet, les DTO d'entrée sont dans dto/request et les DTO de sortie sont dans dto/response. Les mappers transforment les entités en DTO de réponse ou les DTO de requête en entités lorsque c'est nécessaire.

Question possible du professeur	Réponse courte	Fichier à ouvrir
C'est quoi un DTO ?	Un objet qui transporte les données entre Angular et Spring Boot sans exposer l'entité JPA complète.	dto/request/*.java et dto/response/*.java
Où le DTO est-il transformé ?	Dans les mappers, par exemple MembreMapper transforme MembreRequest en Membre et Membre en MembreResponse.	mapper/MembreMapper.java
Pourquoi ne pas renvoyer l'entité directement ?	Pour contrôler les champs exposés, éviter les boucles de relations JPA et stabiliser le contrat API.	mapper/MatchMapper.java, dto/response/MatchDto.java
Où est l'insertion ?	Le controller reçoit le DTO, le mapper crée l'entité si nécessaire, le service applique les règles, le repository sauvegarde.	MembreController.create(), MembreServiceImpl.create(), MembreRepository

12. Architecture frontend détaillée à ajouter au cahier

Le frontend Angular est organisé par responsabilités. Il ne contient aucune requête SQL : toutes les données passent par des services HTTP qui appellent l'API REST du backend. Les pages sont regroupées par espace fonctionnel : accueil, membre et administrateur.

Dossier frontend	Rôle	Exemples
app/routes.ts	Routes racines : accueil, espace membre, espace admin.	path "", path "member", path "admin"
core/api	Services HTTP vers le backend.	auth-api.service.ts, matches-api.service.ts, reservations-api.service.ts
core/auth	Gestion des sessions locales admin et membre.	admin-session.service.ts, member-session.service.ts
core/guards	Protection des routes Angular.	admin.guard.ts, member.guard.ts
core/interceptors	Ajout automatique du JWT dans les requêtes HTTP.	auth-token.interceptor.ts
features/admin/pages	Pages de gestion administrateur.	admin-login.page.ts, admin-sites.page.ts, admin-members.page.ts, admin-matches.page.ts
features/member/pages	Pages du joueur.	member-home.page.ts, member-create-match.page.ts, member-public-matches.page.ts
shared/models	Types TypeScript utilisés par le frontend.	membre.model.ts, match.model.ts, reservation.model.ts, site-terrain.model.ts
shared/utills	Fonctions transverses.	api-error.util.ts

12.1. Services API Angular

Les services API sont la preuve concrète de la communication frontend/backend. Ils utilisent HttpClient et retournent des Observable. C'est cette partie qu'il faut montrer quand on demande comment Angular appelle le backend.

Service Angular	Méthodes principales	Endpoint backend appelé
AuthApiService	loginAdmin(), loginMembre()	POST /api/auth/login, POST /api/membres/login
MatchesApiService	findPublic(), create(), cancel(), convertToPublic()	GET /api/matches/public, POST /api/matches, PATCH /api/matches/{id}/cancel
ReservationsApiService	create(), getByMembre(), cancel()	POST /api/reservations, GET /api/reservations/membre/{id}, PATCH /api/reservations/{id}/cancel
PaielementsApiService	pay(), getByReservation(), getByMembre()	POST /api/paiements/reservation/{id}/pay, GET /api/paiements/membre/{id}
MembresApiService	getAll(), create(), update(), delete()	GET/POST/PUT/DELETE /api/membres
SitesApiService	getAll(), create(), update(), delete()	GET/POST/PUT/DELETE /api/sites
TerrainsApiService	getBySite(), create(), update(), delete()	GET /api/terrains/site/{siteId}, POST/PUT/DELETE /api/terrains
JoursFermetureApiService	getAll(), create(), delete()	GET/POST/DELETE /api/jours-fermeture

12.2. Guards, sessions et interceptor

Les guards frontend améliorent l'expérience utilisateur en empêchant l'accès visuel aux pages protégées. Ils ne remplacent pas la sécurité backend. Le backend reste la référence de sécurité grâce à Spring Security et au JWT.

Élément frontend	Fichier	Rôle
Admin guard	core/guards/admin.guard.ts	Vérifie qu'une session admin existe et que le rôle autorisé est présent.
Member guard	core/guards/member.guard.ts	Vérifie qu'un membre est authentifié avant d'accéder aux pages membre protégées.
Admin session	core/auth/admin-session.service.ts	Stocke le token admin, le rôle GLOBAL/SITE et le siteld dans localStorage.
Member session	core/auth/member-session.service.ts	Stocke le membre connecté et son token ; efface la session admin lors d'une connexion membre.
Interceptor JWT	core/interceptors/auth-token.interceptor.ts	Ajoute Authorization: Bearer <token> aux appels /api.

Flux JWT membre et administrateur

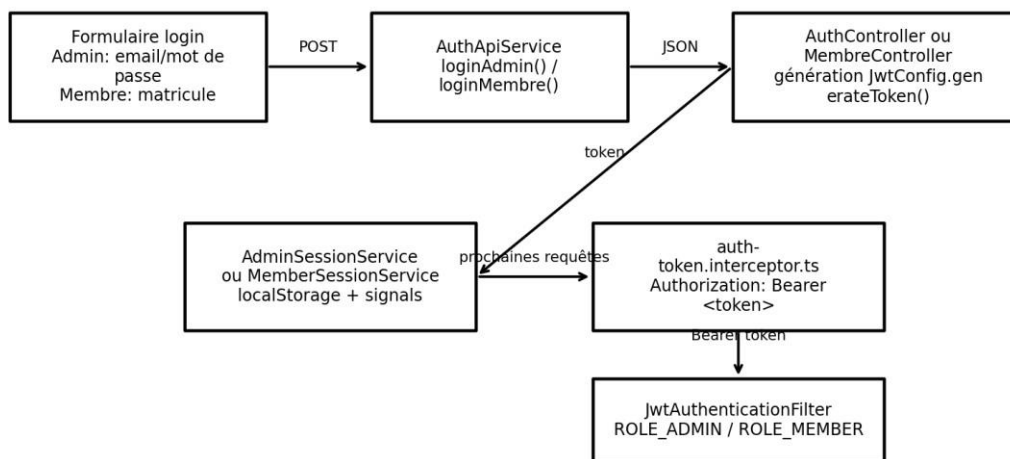


Figure 5 — Flux d'authentification JWT entre Angular et Spring Boot

12.3. Observables et composants réutilisables

Dans Angular, les services HTTP retournent des Observable. Cela se voit dans les services API : loginAdmin(), loginMembre(), create(), getAll(), pay() retournent des Observable typés. Les pages s'abonnent à ces flux ou les consomment pour déclencher l'affichage et les messages d'erreur.

Concernant les composants réutilisables, le projet contient surtout une architecture prête à les accueillir dans shared/components et features/*/components. Les pages actuelles concentrent l'affichage. Pour la défense, il faut expliquer que les éléments réutilisables sont prévus dans shared et que les modèles partagés, les services API, les guards, les sessions et l'utilitaire d'erreur sont déjà réutilisés par plusieurs pages.

Élément réutilisable	Pourquoi il est réutilisable	Localisation
api-error.util.ts	Centralise l'extraction des messages d'erreur API pour plusieurs pages.	shared/utls/api-error.util.ts
Modèles TypeScript	Définissent un contrat commun pour plusieurs services et pages.	shared/models/*.ts
Services API	Réutilisés par toutes les pages qui manipulent la même ressource.	core/api/*.service.ts

Session services	Utilisés par guards, interceptor, pages et navigation.	core/auth/*.service.ts
Guards	Réutilisés dans les routes admin et membre.	core/guards/*.ts

13. Flux fonctionnels complets entre frontend et backend

Cette section complète le cahier avec les transactions fonctionnelles à présenter à l'oral. Chaque flux montre les fichiers dans l'ordre exact : page Angular, service API Angular, contrôleur Spring, service métier, repository et base de données.

Flux	Chaîne technique complète	Règle métier associée
Connexion administrateur	admin-login.page.ts -> AuthApiService.loginAdmin() -> AuthController.login() -> AuthServiceImpl.login() -> AdministrateurRepository -> JwtConfig.generateToken() -> LoginResponse	Authentification admin par e-mail/mot de passe, BCrypt, JWT
Connexion membre	member-home.page.ts -> MemberSessionService.login() -> AuthApiService.loginMembre() -> MembreController.login() -> MembreServiceImpl.getByMatricule() -> JwtConfig.generateToken() -> MembreResponse.token	Authentification membre par matricule, ROLE_MEMBER
Création match	member-create-match.page.ts -> MatchesApiService.create() -> MatchController.createMatch() -> MatchServiceImpl.createMatch() -> TerrainService + MembreService + MatchRepository + ReservationRepository + PaiementRepository	contrôle solde, pénalité, horizon, site, fermeture, horaires, chevauchement, réservation automatique
Réservation	member-public-matches.page.ts -> ReservationsApiService.create() -> ReservationController.create() -> ReservationServiceImpl.create() -> ReservationRepository + PaiementRepository	match complet/annulé, inscription unique, privé/public, SITE, paiement en attente
Paiement	member-payments.page.ts -> PaiementsApiService.pay() -> PaiementController.payCurrentMember() -> PaiementServiceImpl.pay() -> MatchRepository.findByIdForUpdate() -> ReservationService.confirm()	premier payé, premier servi ; confirmation seulement au paiement
Annulation réservation	member-reservations.page.ts -> ReservationsApiService.cancel() -> ReservationController.cancel() -> ReservationServiceImpl.cancel() -> PaiementRepository + MatchService.decrementPlayers()	annulation cohérente, remboursement ou annulation paiement
Conversion automatique	MatchScheduler MatchServiceImpl.checkAndConvertExpiredPrivateMatches() MatchServiceImpl.convertToPublic() MembreServiceImpl.addPenalty() -> MatchRepository + PenaliteRepository	-> match privé incomplet J-1 -> devient public + pénalité
Impayés J-1	MatchScheduler -> PaiementServiceImpl.checkUnpaidBeforeMatch() -> ReservationService.cancel() -> MembreRepository.save()	réservation impayée annulée, solde ajouté à l'organisateur

Réservation puis paiement : principe premier payé, premier servi

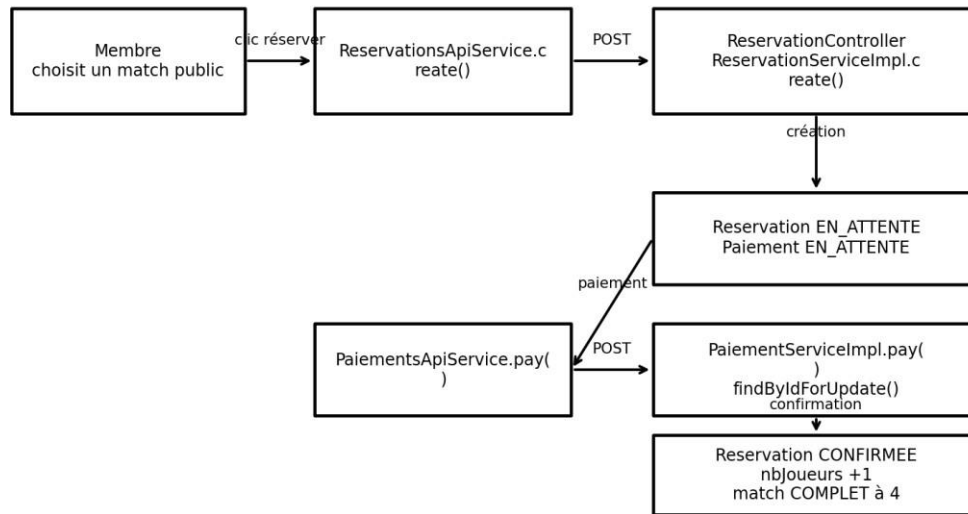


Figure 6 — Flux réservation et paiement avec confirmation de la place

Traitements automatiques quotidiens

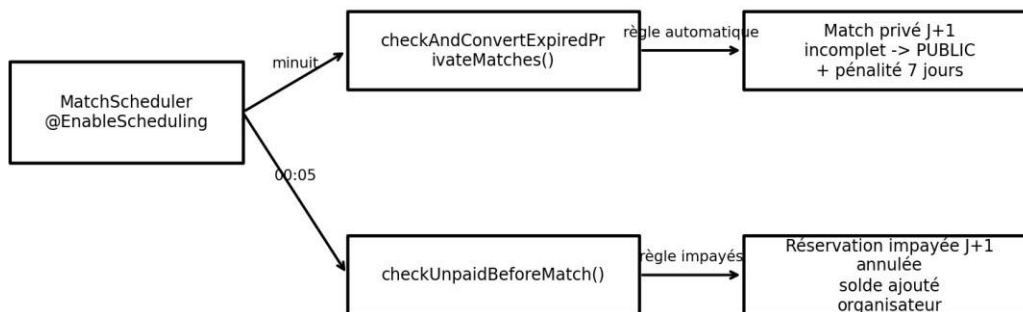


Figure 7 — Flux des traitements automatiques quotidiens

14. Preuves de code par règles métier importantes

Le tableau suivant complète le tableau existant en le rendant plus exploitable pour la défense. Il ne remplace pas les règles RM-01 à RM-75 : il ajoute une localisation directe dans le code pour les règles que le professeur peut demander de montrer.

Règles	Preuve backend	Preuve frontend	À dire à l'oral
RM-01/RM-02 Sites	SiteController + SiteServiceImpl + SiteRepository	admin-sites.page.ts + SitesApiService	CRUD sites et lecture publique

RM-03/RM-05 Terrains	TerrainController + TerrainServiceImpl TerrainRepository	admin-terrains.page.ts + TerrainsApiService	Terrain rattaché à Site, liste par site
-------------------------	--	--	--

Diagramme de séquence — Réservation et paiement d'une place

Application PadelMultiple

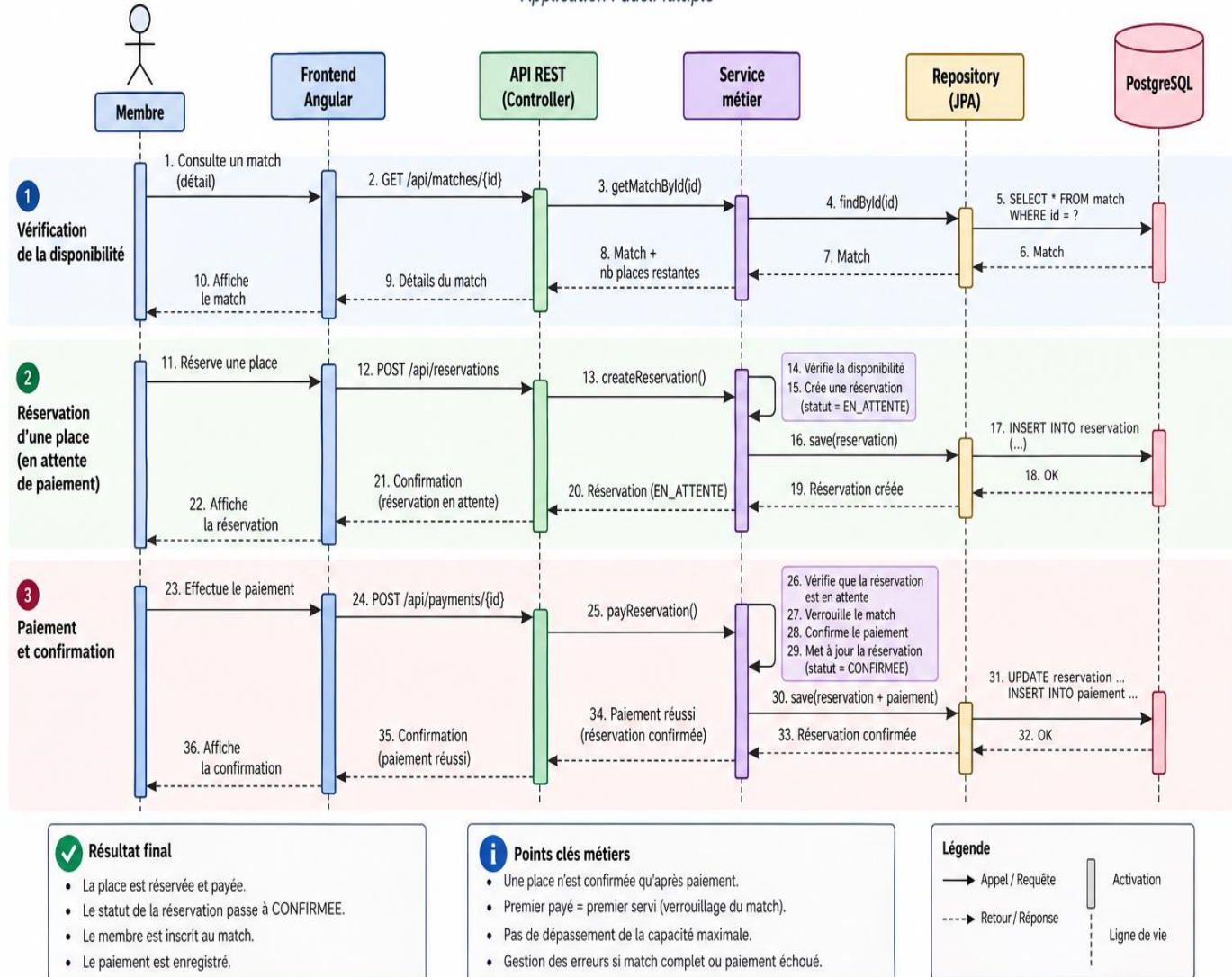


Figure 6 — Diagramme de séquence : réservation et paiement d'une place

PadelMultiple

RM-06 à RM-10 Fermetures	JourFermetureController + JourFermetureRepository + AdminAuthorizationService	admin-jours-fermeture.page.ts + JoursFermetureApiService	Global/site + périmètre admin
RM-11/RM-12 Prix	MatchServiceImpl : MATCH_PRICE = 60.0, MAX_PLAYERS = 4	member-create-match.page.ts + member- payments.page.ts	60 euros, 15 euros par joueur
RM-13 à RM-16 Horizon	MatchServiceImpl.validateBookingDelay()	member-create-match.page.ts	GLOBAL 21, SITE 14, LIBRE 5, pas de date passée
RM-17 Fin automatique	MatchServiceImpl.createMatch() dateFin = dateDebut.plusMinutes(dureeMatch)	member-create-match.page.ts	Le membre choisit le début, le backend calcule la fin
RM-18/RM-19 Chevauchement + pause	MatchRepository.findOverlappingMatches() + isSlotAvailable()	member-create-match.page.ts	Aucun conflit sur un terrain, pause incluse
RM-20 Horaires	MatchServiceImpl.validateSiteOpeningHours()	SitesApiService pour charger horaires côté UI	Match dans les horaires du site
RM-21/RM-22 Fermeture	MatchServiceImpl.validateSiteNotClosed()	admin-jours-fermeture.page.ts	Bloque fermeture globale ou locale
RM-23/RM-24 SITE	validateSiteAccessForOrganizer() + ReservationServiceImpl.create()	member-public-matches.page.ts	Membre SITE limité à son site
RM-25 à RM-28 Capacité/statut	incrementPlayers(), decrementPlayers()	member-public-matches.page.ts	4 joueurs max, complet/planifié
RM-29/RM-30 Organisateur	updateMatch(), cancelMatch()	member-reservations.page.ts / future action UI	Seul l'organisateur modifie/annule
RM-31 à RM-38 Réservation	ReservationServiceImpl.create()	ReservationsApiService.create()	Réservation en attente + paiement en attente
RM-39 à RM-45 Paiement	PaiementServiceImpl.pay()	PaiementsApiService.pay()	Propriétaire, verrou, solde, confirmation
RM-46 Solde	MembreServiceImpl.hasOutstandingBalance()	MembresApiService.hasOutstandingBalance()	Solde > 0 bloque
RM-47 à RM-49 Annulation réservation	ReservationServiceImpl.cancel()	ReservationsApiService.cancel()	Paiement remboursé ou annulé
RM-50 Impayés	PaiementServiceImpl.checkUnpaidBeforeMatch()	Visible ensuite dans profil/paiements	J-1 : impayé reporté à l'organisateur
RM-51 à RM-55 Scheduler	MatchScheduler + MatchServiceImpl.convertToPublic()	member-public-matches.page.ts	Privé incomplet devient public + pénalité
RM-56 à RM-62 Sécurité	SecurityConfig + JwtConfig + JwtAuthenticationFilter + AuthServiceImpl	auth-token.interceptor.ts + guards	JWT sans état, rôles MEMBER/ADMIN
RM-63 à RM-69 Propriété	AdminAuthorizationService + CurrentMemberService	AdminSessionService + MemberSessionService	Ressources filtrées selon utilisateur
RM-70 à RM-75 Validation	DTO request + MembreServiceImpl.validateMatricule()	Formulaires Angular	Bean validation + règles service

14.1. Exemple de flux

Réponse courte : dans ce projet, Angular ne parle jamais directement à la base. Quand l'utilisateur clique, la page Angular appelle un service API. Le service API envoie une requête HTTP vers un contrôleur Spring. Le contrôleur délègue au service métier. Le service applique les règles métier, appelle les repositories JPA, puis les mappers renvoient des DTO JSON au frontend.

Exemple création match : `member-create-match.page.ts` appelle `MatchesApiService.create()`. Le backend reçoit dans `MatchController.createMatch()`. La vraie règle se trouve dans `MatchServiceImpl.createMatch()` : solde impayé, pénalité, fenêtre de réservation, site, fermeture, horaire, chevauchement et création automatique de réservation/paiement. Ensuite `MatchRepository`, `ReservationRepository` et `PaiementRepository` persistent les données.

Exemple paiement : `member-payments.page.ts` appelle `PaiementsApiService.pay()`. `PaiementController` délègue à `PaiementServiceImpl.pay()`. Le service vérifie que le membre est propriétaire, que la réservation est payable, verrouille le match via `MatchRepository.findByIdForUpdate()`, confirme la réservation et incrémente le nombre de

joueurs.

15. Sécurité détaillée : JWT, rôles et autorisations

La sécurité du projet est composée de deux niveaux complémentaires. Côté frontend, les guards empêchent l'utilisateur non connecté d'accéder aux pages protégées. Côté backend, Spring Security décide réellement si une requête HTTP est autorisée. C'est le backend qui fait foi.

Niveau	Fichier	Rôle exact
Frontend	admin.guard.ts	Redirige vers /admin/login si la session admin est absente ou si le rôle ne correspond pas.
Frontend	member.guard.ts	Redirige vers /member si le membre n'est pas connecté.
Frontend	auth-token.interceptor.ts	Ajoute le header Authorization aux appels /api sauf certains appels de login.
Backend	SecurityConfig.java	Déclare les routes publiques, les routes MEMBER, les routes ADMIN et le mode STATELESS.
Backend	JwtConfig.java	Génère le JWT, extrait le subject et le rôle, vérifie la signature.
Backend	JwtAuthenticationFilter.java	Lit Authorization: Bearer, distingue email admin et matricule membre, installe les autorités Spring Security.
Backend	AdminAuthorizationService.java	Vérifie le périmètre métier GLOBAL/SITE après l'authentification technique.
Backend	CurrentMemberService.java	Vérifie qu'un membre agit uniquement sur ses propres ressources.

Le token contient un subject et une claim role. Pour un administrateur, le subject est l'e-mail ; pour un membre, le subject est le matricule. JwtAuthenticationFilter ajoute deux autorités : le rôle métier GLOBAL ou SITE/LIBRE et l'autorité technique ROLE_ADMIN ou ROLE_MEMBER. Cette séparation évite de confondre un membre GLOBAL avec un administrateur GLOBAL.

Point à corriger à l'oral : GLOBAL ne veut pas dire la même chose partout. Un membre GLOBAL est un joueur qui peut réserver sur tous les sites avec 21 jours d'avance. Un administrateur GLOBAL est un gestionnaire qui administre tout le réseau. Le code sépare ces deux mondes avec ROLE_MEMBER et ROLE_ADMIN.

15.1. Routes publiques et routes protégées

Catégorie	Routes principales	Accès
Publiques	/api/auth/**, /swagger-ui/**, /v3/api-docs/**, GET /api/sites/**, GET /api/terrains/**, POST /api/membres/login, GET /api/matches/public	Sans token
Membre	POST /api/matches, PUT /api/matches/{id}, PATCH /api/matches/{id}/cancel, POST /api/reservations, PATCH /api/reservations/{id}/cancel, /api/paiements/**	ROLE_MEMBER
Administrateur	GET /api/matches, GET /api/matches/site/{id}, PATCH /api/matches/{id}/convert-public, CRUD membres, jours de fermeture	ROLE_ADMIN + périmètre métier
Par défaut	Toute route non explicitement ouverte	ROLE_ADMIN

16. Gestion des erreurs et validations

Le cahier doit préciser que la gestion des erreurs est centralisée. Les services ne renvoient pas simplement false : ils lèvent des exceptions métier avec des messages compréhensibles. Le contrôleur global transforme ces

exceptions en réponses HTTP cohérentes.

Classe	Type	Utilisation
BusinessException	Erreur métier	Solde impayé, pénalité active, créneau indisponible, accès refusé, paiement non autorisé.
ResourceNotFoundException	Ressource absente	Membre, match, réservation, paiement, site ou terrain introuvable.
MatchCreationException	Erreur spécialisée	Prévue pour les erreurs de création de match.
ErrorResponse	DTO d'erreur	Structure standard de la réponse d'erreur.
GlobalExceptionHandler	Gestionnaire centralisé	Capture les exceptions et renvoie le bon statut HTTP avec un message.
Bean Validation	Validation d'entrée	@Valid, @NotBlank, @NotNull, @Email sur les DTO request.

Exemples à montrer : MembreRequest impose un matricule, un nom, un prénom et un type de membre. LoginRequest impose l'e-mail et le mot de passe pour l'administrateur. Dans les services, les contrôles métiers complètent ces validations : un format de matricule invalide est refusé dans MembreServiceImpl, un créneau occupé est refusé dans MatchServiceImpl, un paiement déjà effectué est refusé dans PaiementServiceImpl.

Cas d'erreur	Où il est détecté	Message métier
Membre avec solde impayé crée un match	MatchServiceImpl.createMatch()	Vous avez un solde impayé et ne pouvez pas créer de match.
Membre pénalisé réserve	ReservationServiceImpl.create()	Vous avez une pénalité active et ne pouvez pas réserver ce match.
Créneau déjà réservé	MatchServiceImpl.isSlotAvailable()	Ce créneau est déjà réservé sur le terrain.
Paiement par mauvais membre	PaiementServiceImpl.pay()	Seul le membre associé à cette réservation peut payer.
Accès à la réservation d'un autre membre	ReservationServiceImpl.checkReservationOwner()	Accès refusé : cette réservation appartient à un autre membre.
Admin SITE hors périmètre	AdminAuthorizationService.checkSiteAccess()	Un admin SITE ne peut agir que sur son propre site.

17. Complément du modèle de données et des relations JPA

Le modèle de données existant est correct. Le complément ci-dessous précise comment chaque entité est matérialisée dans le code JPA et comment les relations sont utilisées par les règles métier.

Entité	Table	Relations principales	Rôle métier
Site	sites	1-n terrains, 1-n joursFermeture	Définit les horaires, la durée du match, la pause et l'année civile.
Terrain	terrains	n-1 site, 1-n matches	Support physique d'un match ; contient un prix mais le match utilise la constante 60 € dans le service.
Membre	membres	n-1 site optionnel, 1-n matchs organisés, 1-n réservations, 1-n pénalités	Joueur identifié par matricule et profil GLOBAL/SITE/LIBRE.

Administrateur	administrateurs	n-1 site optionnel	Utilisateur d'administration identifié par e-mail et mot de passe hashé.
----------------	-----------------	--------------------	--

Match	matches	n-1 terrain, n-1 organisateur, 1-n réservations	Créneau sportif, public ou privé, planifié/complet/annulé.
Reservation	reservations	n-1 match, n-1 membre, 1-1 paiement	Inscription d'un joueur à un match.
Paiement	paiements	1-1 réservation	Validation financière et confirmation de la place.
Penalite	penalites	n-1 membre	Blocage temporaire après match privé incomplet.

La classe BaseEntity contient les champs communs, notamment l'identifiant généré avec GenerationType.IDENTITY. Les relations en cascade sont utilisées pour garder la cohérence entre les objets liés, par exemple un match et ses réservations ou une réservation et son paiement.

À l'oral, il faut insister sur le fait que JPA est l'outil qui fait le mapping objet-relationnel. Les classes Java représentent les tables, les repositories manipulent les données, et Hibernate génère ou met à jour le schéma avec spring.jpa.hibernate.ddl-auto=update dans application.properties.

18. Catalogue API REST détaillé

Le tableau ci-dessous complète l'aperçu API du cahier avec les méthodes HTTP réellement présentes dans les contrôleurs. Il est utile pour justifier que le cahier correspond aux endpoints du projet.

Méthode	Endpoint	Fonction	Accès
POST	/api/auth/login	Connexion administrateur	Public
POST	/api/membres/login	Connexion membre par matricule	Public
GET	/api/sites, /api/sites/{id}	Consulter les sites	Public
POST/PUT/DELETE	/api/sites	Gérer les sites	Admin
GET	/api/terrains, /api/terrains/site/{siteId}	Consulter les terrains	Public
POST/PUT/DELETE	/api/terrains	Gérer les terrains	Admin
GET/POST/DELETE	/api/jours-fermeture	Gérer les fermetures	Admin
POST	/api/matches	Créer un match	Membre
GET	/api/matches/public	Lister les matchs publics	Public
GET	/api/matches, /api/matches/site/{siteId}	Lister les matchs côté admin	Admin
PUT	/api/matches/{id}	Modifier un match	Membre organisateur
PATCH	/api/matches/{id}/cancel	Annuler un match	Membre organisateur
PATCH	/api/matches/{id}/convert-public	Convertir un match en public	Admin
POST	/api/reservations	Réserver une place	Membre
GET	/api/reservations/membre/{id}	Réservations d'un membre	Membre propriétaire
PATCH	/api/reservations/{id}/cancel	Annuler une réservation	Membre propriétaire
POST	/api/paiements/reservation/{id}/pay	Payer une réservation	Membre propriétaire

GET	/api/paiements/membre/{id}	Historique des paiements	Membre propriétaire
-----	----------------------------	--------------------------	---------------------

Toutes les routes sont préfixées par /api. Côté frontend, environment.apiUrl vaut /api et la fonction apiUrl() concatène ce préfixe avec le chemin de ressource. Le proxy de développement redirige les appels Angular vers le backend Spring Boot sur le port 8080.

19. Tests automatisés et scénarios de validation

Le projet contient des tests backend et frontend qui sont :

Famille de tests	Fichiers	Ce qui est vérifié
Contrôleurs backend	MatchControllerTest, MembreControllerTest, ReservationControllerTest, PaiementControllerTest, JourFermetureControllerTest	Endpoints REST, statuts HTTP, appels aux services, réponses attendues.
Services backend	MatchServiceTest, MembreServiceTest, ReservationServiceTest, PaiementServiceTest, AdminAuthorizationServiceTest	Règles métier principales avec objets simulés et Mockito.
Repositories backend	MatchRepositoryTest, ReservationRepositoryTest, JourFermetureRepositoryTest	Requêtes JPA, chevauchement, recherche par statut/date/site.
Mappers backend	MatchMapperTest, MembreMapperTest, ReservationMapperTest, PaiementMapperTest	Transformation entité -> DTO et cohérence des champs exposés.
Frontend session/guards	admin-session.service.spec.ts, member-session.service.spec.ts, admin.guard.spec.ts, member.guard.spec.ts	Authentification locale, redirections et protection des routes.
Frontend interceptor	auth-token.interceptor.spec.ts	Ajout du header Authorization: Bearer aux appels API.
Frontend pages	member-create-match.page.spec.ts, member-public-matches.page.spec.ts, member-reservations.page.spec.ts, member-admin-members.page.spec.ts	Comportement des pages principales.
Utilitaire erreur	api-error.util.spec.ts	Extraction des messages d'erreur envoyés par le backend.

Commandes de test à inscrire dans le cahier :

Backend

./mvnw test

Frontend cd

frontend npm

test

Pendant l'oral, il faut expliquer que les tests backend utilisent des objets simulés pour tester les contrôleurs et services sans dépendre de toute l'application. Les tests repositories vérifient la couche JPA. Les tests frontend vérifient les

comportements Angular comme les guards, les sessions et l'interceptor JWT.

20. Explicitation brève du projet

Mon projet est séparé en deux applications. Le frontend est en Angular et gère les pages, les formulaires, les routes, les guards, les sessions et les appels HTTP. Le backend est en Spring Boot et applique toutes les règles métier. La base est PostgreSQL et l'accès aux données se fait par JPA via les repositories. Une action utilisateur suit toujours la chaîne : page Angular, service API Angular, contrôleur Spring, service métier, repository JPA, base de données, puis retour DTO JSON.

La sécurité repose sur JWT. Après connexion, Angular stocke le token et l'interceptor l'ajoute dans les requêtes. Spring Security vérifie le token avec JwtAuthenticationFilter. Les routes sont séparées entre ROLE_MEMBER et ROLE_ADMIN, puis les services AdminAuthorizationService et CurrentMemberService vérifient le périmètre exact des ressources.

Les règles métier importantes sont côté backend pour éviter qu'un utilisateur contourne l'interface. Par exemple, la création d'un match vérifie le solde, la pénalité, le délai de réservation, le site, les fermetures, les horaires et les chevauchements. La réservation crée un paiement en attente. La place est confirmée seulement quand le paiement est effectué.

21. Complément explicative des règles métiers

Le cahier initial contient déjà 75 règles. Les règles suivantes ne remplacent pas ces règles : elles les complètent et les rendent plus proches du code livré.

Code complémentaire	Règle ajoutée/nuancée	Justification code
RMC-01	Le prix du terrain existe comme attribut administratif, mais le prix réellement appliqué au match est actuellement fixé par la constante <code>MATCH_PRICE = 60.0</code> .	<code>MatchServiceImpl.createMatch()</code>
RMC-02	La réservation automatique de l'organisateur ne confirme pas encore sa place : elle crée une réservation <code>EN_ATTENTE</code> et un paiement <code>EN_ATTENTE</code> .	<code>createPendingReservationForOrganizer()</code>
RMC-03	Le nombre de joueurs du match représente uniquement les réservations confirmées par paiement.	<code>incrementPlayers()</code> appelé par <code>ReservationService.confirm()</code>
RMC-04	Un paiement est protégé contre la concurrence par un verrou pessimiste sur le match.	<code>MatchRepository.findByIdForUpdate()</code>
RMC-05	Le token d'un membre utilise son matricule comme subject JWT, contrairement à l'administrateur qui utilise son e-mail.	<code>MembreController.login()</code> et <code>JwtAuthenticationFilter</code>
RMC-06	Le rôle métier GLOBAL/SITE/LIBRE d'un membre n'est pas équivalent au rôle technique <code>ROLE_MEMBER</code> .	<code>JwtAuthenticationFilter.authenticate()</code>
RMC-07	Les restrictions de l'admin SITE sont appliquées dans un service dédié et pas seulement dans l'interface.	<code>AdminAuthorizationService</code>
RMC-08	La consultation publique des matchs est limitée aux matchs PUBLIC et PLANIFIE.	<code>MatchServiceImpl.getPublicAvailableMatches()</code>
RMC-09	Les jours de fermeture bloquent la création et la modification d'un match.	<code>validateSiteNotClosed()</code> appelé dans <code>createMatch()</code> et <code>updateMatch()</code>
RMC-10	L'annulation d'une réservation payée transforme le paiement en REMBOURSE ; l'annulation d'une réservation non payée transforme le paiement en ANNULE.	<code>ReservationServiceImpl.cancel()</code>

22. Limites techniques supplémentaires

Ces limites ne remettent pas en cause le fonctionnement du projet, juste pour éviter une contradiction avec le code.

- Le paiement est simulé : aucune intégration Stripe, Bancontact, carte bancaire ou prestataire externe n'est présente.
- Le prix du terrain est stocké mais la logique actuelle de match applique un prix fixe de 60 euros. Une évolution possible serait d'utiliser `terrain.prix` comme prix total du match.
- Le schéma est géré par Hibernate `ddl-auto=update`. Pour un projet en production, il serait préférable d'utiliser Flyway ou Liquibase.
- La clé JWT est définie dans `application.properties`. En production, elle devrait être externalisée dans une variable d'environnement.
- Les notifications e-mail/SMS ne sont pas implémentées.
- Le dossier `shared/components` est préparé mais les composants d'interface réellement factorisés restent limités. La réutilisation actuelle se voit surtout dans les services, modèles, guards, sessions et utilitaires.

- Les guards Angular protègent l'expérience utilisateur, mais la vraie sécurité reste dans Spring Security.

23. **Quelques explications**

La partie existante conserve la description fonctionnelle globale et les règles métier. La présente partie ajoute les localisations de code, les flux techniques, l'architecture détaillée des dossiers, les DTO, les mappers, JPA, les erreurs, les tests, les commandes de lancement et les points à corriger ou à nuancer pendant la défense.

le frontend Angular gère l'affichage et les appels HTTP ; le backend Spring Boot applique les règles métier et sécurise les routes ; JPA persiste les entités en base PostgreSQL ; les DTO et mappers assurent un échange propre entre l'API et l'interface ; les tests valident les contrôleurs, services, repositories, mappers et comportements Angular.

24. Checklist finale futur

1. Ouvrir le projet dans IntelliJ et montrer le package réel `com.padell.padell`.
2. Montrer `MatchServiceImpl.createMatch()` pour les règles de création de match.
3. Montrer `ReservationServiceImpl.create()` pour les règles de réservation.
4. Montrer `PaiementServiceImpl.pay()` et `MatchRepository.findByIdForUpdate()` pour premier payé/premier servi.
5. Montrer `SecurityConfig` et `JwtAuthenticationFilter` pour la sécurité JWT.
6. Montrer `auth-token.interceptor.ts`, `admin.guard.ts` et `member.guard.ts` pour la sécurité frontend.
7. Montrer `entity/*.java` et `repository/*.java` pour JPA.
8. Montrer `dto/request`, `dto/response` et `mapper/*.java` pour DTO et mapping.
9. Montrer `GlobalExceptionHandler` et `BusinessException` pour la gestion des erreurs.
10. Montrer les tests backend et frontend si le professeur demande les preuves de validation.
- 11.